RTIC

The hardware accelerated Rust RTOS

A concurrency framework for building real-time systems

Contents

1. Preface

This book contains user level documentation for the Real-Time Interrupt-driven Concurrency (RTIC) framework. The API reference is available here.

This is the documentation for RTIC v2.x.

Older releases: RTIC v1.x | RTIC v0.5.x (unsupported) | RTFM v0.4.x (unsupported)

1.1. Is RTIC an RTOS?

A common question is whether RTIC is an RTOS or not, and depending on your background the answer may vary. From RTIC's developers point of view; RTIC is a hardware accelerated RTOS that utilizes the hardware such as the NVIC on Cortex-M MCUs, CLIC on RISC-V etc. to perform scheduling, rather than the more classical software kernel.

Another common view from the community is that RTIC is a concurrency framework as there is no software kernel and that it relies on external HALs.

1.2. RTIC - The Past, current and Future

This section gives a background to the RTIC model. Feel free to skip to section [RTIC the model] for a TL;DR.

The RTIC framework takes the outset from real-time systems research at Luleå University of Technology (LTU) Sweden. RTIC is inspired by the concurrency model of the Timber language, the RTFM-SRP based scheduler, the RTFM-core language and Abstract Timer implementation. For a full list of related research see RTFM and RTIC publications.

1.3. Stack Resource Policy based Scheduling

Stack Resource Policy (SRP) based concurrency and resource management is at heart of the RTIC framework. The SRP model itself extends on Priority Inheritance Protocols, and provides a set of outstanding properties for single core scheduling. To name a few:

- preemptive deadlock and race-free scheduling
- resource efficiency
  - tasks execute on a single shared stack
  - tasks run-to-completion with wait free access to shared resources
- predictable scheduling, with bounded priority inversion by a single (named) critical section
- theoretical underpinning amenable to static analysis (e.g., for task response times and overall schedulability)

SRP comes with a set of system-wide requirements: - each task is associated a static priority, - tasks execute on a single-core,
- tasks must be run-to-completion, and - resources must be claimed/locked in LIFO order.

1.4. SRP analysis

SRP based scheduling requires the set of static priority tasks and their access to shared resources to be known in order to compute a static *ceiling* (π) for each resource. The static resource *ceiling* π(r) reflects the maximum static priority of any task that accesses the resource `r`.

1.4.1. Example

Assume two tasks `A` (with priority `p(A) = 2`) and `B` (with priority `p(B) = 4`) both accessing the shared resource `R`. The static ceiling of `R` is 4 (computed from π`(R) = max(p(A) = 2, p(B) = 4) = 4`).

A graph representation of the example:

```mermaid
graph LR
    A["p(A) = 2"] --> R
    B["p(B) = 4"] --> R
    R["𝝅(R) = 4"]
```

1.5. RTIC the hardware accelerated real-time scheduler

SRP itself is compatible with both dynamic and static priority scheduling. For the implementation of RTIC we leverage on the underlying hardware for accelerated static priority scheduling.

In the case of the `ARM Cortex-M` architecture, each interrupt vector entry `v[i]` is associated a function pointer (`v[i].fn`), a static priority (`v[i].priority`), an enabled- (`v[i].enabled`) and a pending-bit (`v[i].pending`).

An interrupt `i` is scheduled (run) by the hardware under the conditions: 1. is `pending` and `enabled` and has a priority higher than the (optional `BASEPRI`) register, and 1. has the highest priority among interrupts meeting 1.

The first condition (1) can be seen as a filter, allowing RTIC to take control over which tasks should be allowed to start (and which should be prevented from starting).

The SRP model for single-core static scheduling, on the other hand, states that a task should be scheduled (run) under the conditions: 1. it is `requested` to run and has a static priority higher than the current system ceiling (Π) 1. it has the highest static priority among tasks meeting 1.

The similarities are striking and it is not by chance/luck/coincidence. The hardware was cleverly designed with real-time scheduling in mind.

In order to map the SRP scheduling onto the hardware we need to take a closer look at the system ceiling (Π). Under SRP Π is computed as the maximum priority ceiling of the currently held resources, and will thus change dynamically during the system operation.

1.5.1. Example

Assume the task model above. Starting from an idle system, Π is 0, (no task is holding any resource). Assume that `A` is requested for execution, it will immediately be scheduled. Assume that `A` claims (locks) the resource `R`. During the claim (lock of `R`) any request `B` will be blocked from starting (by Π = `max(`π`(R) = 4) = 4`, `p(B) = 4`, thus SRP scheduling condition 1 is not met).

1.6. Mapping

The mapping of static priority SRP based scheduling to the Cortex M hardware is straightforward:

- each task `t` are mapped to an interrupt vector index `i` with a corresponding function `v[i].fn = t` and given the static priority `v[i].priority = p(t)`.
- the current system ceiling is mapped to the `BASEPRI` register or implemented through masking the interrupt enable bits accordingly.

1.6.1. Example

For the running example, a snapshot of the ARM Cortex M Nested Vectored Interrupt Controller (NVIC) may have the following configuration (after task `A` has been pended for execution.)

Index	Fn	Priority	Enabled	Pending
0	A	2	true	true
1	B	4	true	false

(As discussed later, the assignment of interrupt and exception vectors is up to the user.)

A claim (lock(r)) will change the current system ceiling ($\boldsymbol{\Pi}$) and can be implemented as a *named* critical section: - old_ceiling = $\boldsymbol{\Pi}$, $\boldsymbol{\Pi}$ = $\boldsymbol{\pi}$(r)
- execute code within critical section - $\boldsymbol{\Pi}$ = old_ceiling

This amounts to a resource protection mechanism, requiring only two machine instructions on enter and one on exit the critical section, for managing the `BASEPRI` register. For architectures lacking `BASEPRI`, we can implement the system ceiling through a set of machine instructions for disabling/enabling interrupts on entry/exit for the named critical section. The number of machine instructions vary depending on the number of mask registers that needs to be updated (a single machine operation can operate on up to 32 interrupts, so for the M0/M0+ architecture a single instruction suffice). RTIC will determine the ceiling values and masking constants at compile time, thus all operations is in Rust terms zero-cost.

In this way RTIC fuses SRP based preemptive scheduling with a zero-cost hardware accelerated implementation, resulting in “best in class” guarantees and performance.

Given that the approach is dead simple, how come SRP and hardware accelerated scheduling is not adopted by any other mainstream RTOS?

The answer is simple, the commonly adopted threading model does not lend itself well to static analysis - there is no known way to extract the task/resource dependencies from the source code at compile time (thus ceilings cannot be efficiently computed and the LIFO resource locking requirement cannot be ensured). Thus, SRP based scheduling is in the general case out of reach for any thread based RTOS.

1.7. RTIC into the Future

Asynchronous programming in various forms are getting increased popularity and language support. Rust natively provides an `async`/`await` API for cooperative multitasking and the compiler generates the necessary boilerplate for storing and retrieving execution contexts (i.e., managing the set of local variables that spans each `await`).

The Rust standard library provides collections for dynamically allocated data-structures which are useful to manage execution contexts at run-time. However, in the setting of resource constrained real-time systems, dynamic allocations are problematic (both regarding performance and reliability - Rust runs into a *panic* on an out-of-memory condition). Thus, static allocation is the preferable approach!

From a modelling perspective `async/await` lifts the run-to-completion requirement of SRP, and each section of code between two yield points (`await`s) can be seen as an individual task. The compiler will reject any attempt to `await` while holding a resource (not doing so would break the strict LIFO requirement on resource usage under SRP).

So with the technical stuff out of the way, what does `async/await` bring to the table?

The answer is - improved ergonomics! A recurring use case is to have task perform a sequence of requests and then await their results in order to progress. Without `async`/`await` the programmer would be forced to split the task into individual sub-tasks and maintain some sort of state encoding (and manually progress by selecting sub-task). Using `async/await` each yield point (`await`) essentially represents a state, and the progression mechanism is built automatically for you at compile time by means of `Futures`.

Rust `async`/`await` support is still incomplete and/or under development. Nevertheless, it covers most common use cases and can be considered production ready.

An important property is that futures are composable, thus you can await either, all, or any combination of possible futures (allowing e.g., timeouts and/or asynchronous errors to be promptly handled).

1.8. RTIC the model

An RTIC `app` is a declarative and executable system model for single-core applications, defining a set of (`local` and `shared`) resources operated on by a set of (`init`, `idle`, *hardware* and *software*) tasks. In short the `init` task runs before any other task returning a set of resources (`local` and `shared`). Tasks run preemptively based on their associated static priority, `idle` has the lowest priority (and can be used for background work, and/or to put the system to sleep until woken by some event). Hardware tasks are bound to underlying hardware interrupts, while software tasks are scheduled by asynchronous executors (one for each software task priority).

At compile time the task/resource model is analyzed under SRP and executable code generated with the following outstanding properties:

- guaranteed race-free resource access and deadlock-free execution on a single-shared stack (thanks to SRP)
  - hardware task scheduling is performed directly by the hardware, and
  - software task scheduling is performed by auto generated async executors tailored to the application.

The RTIC API design ensures that both SRP requirements and Rust soundness rules are upheld at all times, thus the executable model is correct by construction. Overall, the generated code infers no additional overhead in comparison to a handwritten implementation, thus in Rust terms RTIC offers a zero-cost abstraction to concurrency.

2. Starting a new project

A recommendation when starting a RTIC project from scratch is to follow RTIC's `defmt-app-template`.

If you are targeting ARMv6-M or ARMv8-M-base architecture, check out the section Target Architecture for more information on hardware limitations to be aware of.

This will give you an RTIC application with support for RTT logging with `defmt` and stack overflow protection using `flip-link`. There is also a multitude of examples provided by the community:

For inspiration, you may look at the RTIC examples.

2.1. RTIC on RISC-V devices

Even though RTIC was initially developed for ARM Cortex-M, it is possible to use RTIC on RISC-V devices. However, the RISC-V ecosystem is more heterogeneous. To tackle this issue, currently, RTIC implements three different backends:

- **`riscv-esp32c3-backend`**: This backend provides support for the ESP32-C3 SoC. In these devices, RTIC is very similar to its Cortex-M counterpart.
- **`riscv-esp32c6-backend`**: This backend provides support for the ESP32-C6 SoC. In these devices, RTIC is very similar to its Cortex-M counterpart.
- **`riscv-mecall-backend`**: This backend provides support for **any** RISC-V device. In this backend, pending tasks trigger Machine Environment Call exceptions. The handler for this exception source dispatches pending tasks according to their priority. The behavior of this backend is equivalent to `riscv-clint-backend`. The main difference of this backend is that all the tasks **must be** software tasks. Additionally, it is not required to provide a list of dispatchers in the `#[app]` attribute, as RTIC will generate them at compile time.
- **`riscv-clint-backend`**: This backend supports devices with a CLINT peripheral. It is equivallent to `riscv-mecall-backend`, but instead of triggering exceptions, it triggers software interrupts via the `MSIP` register of the CLINT.

3. RTIC by example

This part of the book introduces the RTIC framework to new users by walking them through examples of increasing complexity.

All examples in this part of the book are part of the RTIC repository, found in the `examples` directory. The examples are runnable on QEMU (emulating a Cortex M3 target), thus no special hardware required to follow along.

3.1. Running an example

To run the examples with QEMU you will need the `qemu-system-arm` program. Check the embedded Rust book for instructions on how to set up an embedded development environment that includes QEMU.

To run the examples found in `examples/` locally using QEMU:

```
cargo xtask qemu
```

This runs all of the examples against the default `thumbv7m-none-eabi` device `lm3s6965`.

To limit which examples are being run, use the flag `--example <example name>`, the name being the filename of the example.

Assuming dependencies in place, running:

```
$ cargo xtask qemu --example locals
```
Shell

Yields this output:

```
   Finished dev [unoptimized + debuginfo] target(s) in 0.07s
    Running `target/debug/xtask qemu --example locals`
INFO  xtask::run > QEMU run for platform: Lm3s6965, backend: Thumbv7
INFO  xtask::run > 👟 Build example locals (thumbv7m-none-eabi, release, "thumbv7-backend", in examples/lm3s6965)
INFO  xtask::run > ✅ Success.
INFO  xtask::run > 👟 Run example locals in QEMU (thumbv7m-none-eabi, release, "thumbv7-backend", in examples/lm3s6965)
INFO  xtask::run > ✅ Success.
INFO  xtask::results > ✅ Success: Build example locals (thumbv7m-none-eabi, release, "thumbv7-backend", in examples/lm3s6965)
INFO  xtask::results > ✅ Success: Run example locals in QEMU (thumbv7m-none-eabi, release, "thumbv7-backend", in examples/lm3s6965)
INFO  xtask::results > 🚀🚀🚀 All tasks succeeded 🚀🚀🚀
```
Shell

It is great that examples are passing and this is part of the RTIC CI setup too, but for the purposes of this book we must add the `--verbose` flag, or `-v` for short to see the actual program output:

```
❯ cargo xtask qemu --verbose --example locals
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.04s
     Running `target/debug/xtask qemu --verbose --example locals`
DEBUG xtask > Stderr of child processes is inherited: false
DEBUG xtask > Partial features: false
INFO  xtask::run > QEMU run for platform: Lm3s6965, backend: Thumbv7
```
Shell

```
 INFO  xtask::run > 👟 Build example locals (thumbv7m-none-eabi, release, "thumbv7-
backend", in examples/lm3s6965)
 INFO  xtask::run > ✅ Success.
 INFO  xtask::run > 👟 Run example locals in QEMU (thumbv7m-none-eabi, release,
"thumbv7-backend", in examples/lm3s6965)
 INFO  xtask::run > ✅ Success.
 INFO  xtask::results > ✅ Success: Build example locals (thumbv7m-none-eabi,
release, "thumbv7-backend", in examples/lm3s6965)
    cd examples/lm3s6965 && cargo build --target thumbv7m-none-eabi --features
    thumbv7-backend --release --example locals
 DEBUG xtask::results >
cd examples/lm3s6965 && cargo build --target thumbv7m-none-eabi --features thumbv7-
backend --release --example locals
Stderr:
    Finished `release` profile [optimized] target(s) in 0.03s
 INFO  xtask::results > ✅ Success: Run example locals in QEMU (thumbv7m-none-eabi,
release, "thumbv7-backend", in examples/lm3s6965)
    cd examples/lm3s6965 && cargo run --target thumbv7m-none-eabi --features
    thumbv7-backend --release --example locals
 DEBUG xtask::results >
cd examples/lm3s6965 && cargo run --target thumbv7m-none-eabi --features thumbv7-
backend --release --example locals
Stdout:
bar: local_to_bar = 1
foo: local_to_foo = 1
idle: local_to_idle = 1

Stderr:
    Finished `release` profile [optimized] target(s) in 0.03s
     Running `qemu-system-arm -cpu cortex-m3 -machine lm3s6965evb -nographic -
     semihosting-config enable=on,target=native -kernel target/thumbv7m-none-eabi/
     release/examples/locals`
Timer with period zero, disabling

 INFO  xtask::results > 🚀🚀🚀 All tasks succeeded 🚀🚀🚀
```

Look for the content following `Stdout:` towards the end ouf the output, the program output should have these lines:

```
bar: local_to_bar = 1                                                    Plain Text
foo: local_to_foo = 1
idle: local_to_idle = 1
```

NOTE: For other useful options to `cargo xtask`, see:

```
cargo xtask qemu --help
```

The `--platform` flag allows changing which device examples are run on, currently `lm3s6965` is the best supported, work is ongoing to increase support for other devices, including both ARM and RISC-V

3.2. The `#[app]` attribute and an RTIC application

3.2.1. Requirements on the `app` attribute

All RTIC applications use the `app` attribute (`#[app(..)]`). This attribute only applies to a `mod`-item containing the RTIC application.

The `app` attribute has a mandatory `device` argument that takes a *path* as a value. This must be a full path pointing to a *peripheral access crate* (PAC) generated using `svd2rust` **v0.14.x** or newer.

The `app` attribute will expand into a suitable entry point and thus replaces the use of the `cortex_m_rt::entry` attribute.

3.2.2. Structure and zero-cost concurrency

An RTIC `app` is an executable system model for single-core applications, declaring a set of `local` and `shared` resources operated on by a set of `init`, `idle`, *hardware* and *software* tasks.

- `init` runs before any other task, and returns the `local` and `shared` resources.
- Tasks (both hardware and software) run preemptively based on their associated static priority.
- Hardware tasks are bound to underlying hardware interrupts.
- Software tasks are schedulied by an set of asynchronous executors, one for each software task priority.
- `idle` has the lowest priority, and can be used for background work, and/or to put the system to sleep until it is woken by some event.

At compile time the task/resource model is analyzed under the Stack Resource Policy (SRP) and executable code generated with the following outstanding properties:

- Guaranteed race-free resource access and deadlock-free execution on a single-shared stack.
- Hardware task scheduling is performed directly by the hardware.
- Software task scheduling is performed by auto generated async executors tailored to the application.

Overall, the generated code infers no additional overhead in comparison to a hand-written implementation, thus in Rust terms RTIC offers a zero-cost abstraction to concurrency.

3.2.3. Priority

Priorities in RTIC are specified using the `priority = N` (where N is a positive number) argument passed to the `#[task]` attribute. All `#[task]`s can have a priority. If the priority of a task is not specified, it is set to the default value of 0.

Priorities in RTIC follow a higher value = more important scheme. For examples, a task with priority 2 will preempt a task with priority 1.

3.2.4. An RTIC application example

To give a taste of RTIC, the following example contains commonly used features. In the following sections we will go through each feature in detail.

```rust
//! examples/common.rs

#![no_main]
#![no_std]
#![deny(warnings)]
```

```
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965, dispatchers = [UART0, UART1])]
12  mod app {
13      use cortex_m_semihosting::{debug, hprintln};
14
15      #[shared]
16      struct Shared {}
17
18      #[local]
19      struct Local {
20          local_to_foo: i64,
21          local_to_bar: i64,
22          local_to_idle: i64,
23      }
24
25      // `#[init]` cannot access locals from the `#[local]` struct as they are
        initialized here.
26      #[init]
27      fn init(_: init::Context) -> (Shared, Local) {
28          foo::spawn().unwrap();
29          bar::spawn().unwrap();
30
31          (
32              Shared {},
33              // initial values for the `#[local]` resources
34              Local {
35                  local_to_foo: 0,
36                  local_to_bar: 0,
37                  local_to_idle: 0,
38              },
39          )
40      }
41
42      // `local_to_idle` can only be accessed from this context
43      #[idle(local = [local_to_idle])]
44      fn idle(cx: idle::Context) -> ! {
45          let local_to_idle = cx.local.local_to_idle;
46          *local_to_idle += 1;
47
48          hprintln!("idle: local_to_idle = {}", local_to_idle);
49
50          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
```

```
51
52          // error: no `local_to_foo` field in `idle::LocalResources`
53          // _cx.local.local_to_foo += 1;
54
55          // error: no `local_to_bar` field in `idle::LocalResources`
56          // _cx.local.local_to_bar += 1;
57
58          loop {
59              cortex_m::asm::nop();
60          }
61      }
62
63      // `local_to_foo` can only be accessed from this context
64      #[task(local = [local_to_foo], priority = 1)]
65      async fn foo(cx: foo::Context) {
66          let local_to_foo = cx.local.local_to_foo;
67          *local_to_foo += 1;
68
69          // error: no `local_to_bar` field in `foo::LocalResources`
70          // cx.local.local_to_bar += 1;
71
72          hprintln!("foo: local_to_foo = {}", local_to_foo);
73      }
74
75      // `local_to_bar` can only be accessed from this context
76      #[task(local = [local_to_bar], priority = 1)]
77      async fn bar(cx: bar::Context) {
78          let local_to_bar = cx.local.local_to_bar;
79          *local_to_bar += 1;
80
81          // error: no `local_to_foo` field in `bar::LocalResources`
82          // cx.local.local_to_foo += 1;
83
84          hprintln!("bar: local_to_bar = {}", local_to_bar);
85      }
86  }
```

3.3. Hardware tasks

At its core RTIC is using a hardware interrupt controller (ARM NVIC on cortex-m) to schedule and start execution of tasks. All tasks except `pre-init` (a hidden "task"), `#[init]` and `#[idle]` run as interrupt handlers.

To bind a task to an interrupt, use the `#[task]` attribute argument `binds = InterruptName`. This task then becomes the interrupt handler for this hardware interrupt vector.

All tasks bound to an explicit interrupt are called *hardware tasks* since they start execution in reaction to a hardware event.

Specifying a non-existing interrupt name will cause a compilation error. The interrupt names are commonly defined by PAC or HAL crates.

Any available interrupt vector should work. Specific devices may bind specific interrupt priorities to specific interrupt vectors outside user code control. See for example the nRF “softdevice”.

Beware of using interrupt vectors that are used internally by hardware features; RTIC is unaware of such hardware specific details.

3.3.1. Example

The example below demonstrates the use of the `#[task(binds = InterruptName)]` attribute to declare a hardware task bound to an interrupt handler.

```rust
//! examples/hardware.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965)]
mod app {
    use cortex_m_semihosting::{debug, hprintln};
    use lm3s6965::Interrupt;

    #[shared]
    struct Shared {}

    #[local]
    struct Local {}

    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        // Pends the UART0 interrupt but its handler won't run until *after*
        // `init` returns because interrupts are disabled
        rtic::pend(Interrupt::UART0); // equivalent to NVIC::pend

        hprintln!("init");

        (Shared {}, Local {})
    }

    #[idle]
    fn idle(_: idle::Context) -> ! {
        // interrupts are enabled again; the `UART0` handler runs at this point
```

```
36
37          hprintln!("idle");
38
39          // Some backends provide a manual way of pending an
40          // interrupt.
41          rtic::pend(Interrupt::UART0);
42
43          loop {
44              cortex_m::asm::nop();
45              debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
46          }
47      }
48
49      #[task(binds = UART0, local = [times: u32 = 0])]
50      fn uart0(cx: uart0::Context) {
51          // Safe access to local `static mut` variable
52          *cx.local.times += 1;
53
54          hprintln!(
55              "UART0 called {} time{}",
56              *cx.local.times,
57              if *cx.local.times > 1 { "s" } else { "" }
58          );
59      }
60  }
```

```
$ cargo xtask qemu --verbose --example hardware
```
Shell

```
init
UART0 called 1 time
idle
UART0 called 2 times
```
Plain Text

3.4. Software tasks & spawn

The RTIC concept of a software task shares a lot with that of hardware tasks. The core difference is that a software task is not explicitly bound to a specific interrupt vector, but rather bound to a "dispatcher" interrupt vector running at the intended priority of the software task (see below).

Similarly to *hardware* tasks, the `#[task]` attribute used on a function declare it as a task. The absence of a `binds = InterruptName` argument to the attribute declares the function as a *software task*.

The static method `task_name::spawn()` spawns (starts) a software task and given that there are no higher priority tasks running the task will start executing directly.

The *software* task itself is given as an `async` Rust function, which allows the user to optionally `await` future events. This allows to blend reactive programming (by means of *hardware* tasks) with sequential programming (by means of *software* tasks).

While *hardware* tasks are assumed to run-to-completion (and return), *software* tasks may be started (`spawned`) once and run forever, on the condition that any loop (execution path) is broken by at least one `await` (yielding operation).

3.4.1. Dispatchers

All *software* tasks at the same priority level share an interrupt handler acting as an async executor dispatching the software tasks. This list of dispatchers, `dispatchers = [FreeInterrupt1, FreeInterrupt2, ...]` is an argument to the `#[app]` attribute, where you define the set of free and usable interrupts.

Each interrupt vector acting as dispatcher gets assigned to one priority level meaning that the list of dispatchers need to cover all priority levels used by software tasks.

Example: The `dispatchers =` argument needs to have at least 3 entries for an application using three different priorities for software tasks.

The framework will give a compilation error if there are not enough dispatchers provided, or if a clash occurs between the list of dispatchers and interrupts bound to *hardware* tasks.

See the following example:

```Rust
//! examples/spawn.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965, dispatchers = [SSI0])]
mod app {
    use cortex_m_semihosting::{debug, hprintln};

    #[shared]
    struct Shared {}

    #[local]
    struct Local {}

    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        hprintln!("init");
        foo::spawn().unwrap();

        (Shared {}, Local {})
    }

    #[task]
```

```
30      async fn foo(_: foo::Context) {
31          hprintln!("foo");
32
33          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
34      }
35  }
```

```
$ cargo xtask qemu --verbose --example spawn
```
Shell

```
init
foo
```
Plain Text

You may `spawn` a *software* task again, given that it has run-to-completion (returned).

In the below example, we `spawn` the *software* task `foo` from the `idle` task. Since the priority of the *software* task is 1 (higher than `idle`), the dispatcher will execute `foo` (preempting `idle`). Since `foo` runs-to-completion. It is ok to `spawn` the `foo` task again.

Technically the async executor will `poll` the `foo` *future* which in this case leaves the *future* in a *completed* state.

```
1   //! examples/spawn_loop.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965, dispatchers = [SSI0])]
12  mod app {
13      use cortex_m_semihosting::{debug, hprintln};
14
15      #[shared]
16      struct Shared {}
17
18      #[local]
19      struct Local {}
20
21      #[init]
22      fn init(_: init::Context) -> (Shared, Local) {
23          hprintln!("init");
24
25          (Shared {}, Local {})
26      }
27
28      #[idle]
```
Rust

```
    fn idle(_: idle::Context) -> ! {
        for _ in 0..3 {
            foo::spawn().unwrap();
            hprintln!("idle");
        }
        debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
        loop {}
    }

    #[task(priority = 1)]
    async fn foo(_: foo::Context) {
        hprintln!("foo");
    }
}
```

```
$ cargo xtask qemu --verbose --example spawn_loop
```
Shell

```
init
foo
idle
foo
idle
foo
idle
```
Plain Text

An attempt to `spawn` an already spawned task (running) task will result in an error. Notice, the that the error is reported before the `foo` task is actually run. This is since, the actual execution of the *software* task is handled by the dispatcher interrupt (`SSI0`), which is not enabled until we exit the `init` task. (Remember, `init` runs in a critical section, i.e. all interrupts being disabled.)

Technically, a `spawn` to a *future* that is not in *completed* state is considered an error.

```rust
//! examples/spawn_err.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965, dispatchers = [SSI0])]
mod app {
    use cortex_m_semihosting::{debug, hprintln};

    #[shared]
    struct Shared {}
```
Rust

```
17
18      #[local]
19      struct Local {}
20
21      #[init]
22      fn init(_: init::Context) -> (Shared, Local) {
23          hprintln!("init");
24          foo::spawn().unwrap();
25          match foo::spawn() {
26              Ok(_) => {}
27              Err(()) => hprintln!("Cannot spawn a spawned (running) task!"),
28          }
29
30          (Shared {}, Local {})
31      }
32
33      #[task]
34      async fn foo(_: foo::Context) {
35          hprintln!("foo");
36
37          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
38      }
39  }
```

```
$ cargo xtask qemu --verbose --example spawn_err
```
Shell

```
init
Cannot spawn a spawned (running) task!
foo
```
Plain Text

3.4.2. Passing arguments

You can also pass arguments at spawn as follows.

```
1   //! examples/spawn_arguments.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965, dispatchers = [SSI0])]
12  mod app {
13      use cortex_m_semihosting::{debug, hprintln};
14
```
Rust

```
15      #[shared]
16      struct Shared {}
17
18      #[local]
19      struct Local {}
20
21      #[init]
22      fn init(_: init::Context) -> (Shared, Local) {
23          foo::spawn(1, 1).unwrap();
24          assert!(foo::spawn(1, 4).is_err()); // The capacity of `foo` is reached
25
26          (Shared {}, Local {})
27      }
28
29      #[task]
30      async fn foo(_c: foo::Context, x: i32, y: u32) {
31          hprintln!("foo {}, {}", x, y);
32          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
33      }
34  }
```

```
$ cargo xtask qemu --verbose --example spawn_arguments
```
Shell

```
foo 1, 1
```
Plain Text

3.4.3. Divergent tasks

A task can have one of two signatures: `async fn({name}::Context, ..)` or `async fn({name}::Context, ..) -> !`. The latter defines a *divergent* task — one that never returns. The key advantage of divergent tasks is that they receive a `'static` context, and `local` resources have `'static` lifetime. Additionally, using this signature makes the task's intent explicit, clearly distinguishing between short-lived tasks and those that run indefinitely. Be mindful not to starve other tasks at the same priority level by ensuring you yield control with `.await`.

3.4.4. Priority zero tasks

In RTIC tasks run preemptively to each other, with priority zero (0) the lowest priority. You can use priority zero tasks for background work, without any strict real-time requirements.

Conceptually, one can see such tasks as running in the `main` thread of the application, thus the resources associated are not required the Send bound.

```
1   //! examples/zero-prio-task.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
```
Rust

```
9   use core::marker::PhantomData;
10  use panic_semihosting as _;
11
12  /// Does not impl send
13  pub struct NotSend {
14      _0: PhantomData<*const ()>,
15  }
16
17  #[rtic::app(device = lm3s6965, peripherals = true)]
18  mod app {
19      use super::NotSend;
20      use core::marker::PhantomData;
21      use cortex_m_semihosting::{debug, hprintln};
22
23      #[shared]
24      struct Shared {
25          x: NotSend,
26      }
27
28      #[local]
29      struct Local {
30          y: NotSend,
31      }
32
33      #[init]
34      fn init(_cx: init::Context) -> (Shared, Local) {
35          hprintln!("init");
36
37          async_task::spawn().unwrap();
38          async_task2::spawn().unwrap();
39
40          (
41              Shared {
42                  x: NotSend { _0: PhantomData },
43              },
44              Local {
45                  y: NotSend { _0: PhantomData },
46              },
47          )
48      }
49
50      #[task(priority = 0, shared = [x], local = [y])]
51      async fn async_task(_: async_task::Context) {
52          hprintln!("hello from async");
53      }
54
```

```
55      #[task(priority = 0, shared = [x])]
56      async fn async_task2(_: async_task2::Context) {
57          hprintln!("hello from async2");
58
59          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
60      }
61  }
```

```
$ cargo xtask qemu --verbose --example zero-prio-task
```
Shell

```
init
hello from async
hello from async2
```
Plain Text

> **Notice**: *software* task at zero priority cannot co-exist with the [idle] task. The reason is that `idle` is running as a non-returning Rust function at priority zero. Thus there would be no way for an executor at priority zero to give control to *software* tasks at the same priority.

Application side safety: Technically, the RTIC framework ensures that `poll` is never executed on any *software* task with *completed* future, thus adhering to the soundness rules of async Rust.

3.5. Resource usage

The RTIC framework manages shared and task local resources allowing persistent data storage and safe accesses without the use of `unsafe` code.

RTIC resources are visible only to functions declared within the `#[app]` module and the framework gives the user complete control (on a per-task basis) over resource accessibility.

Declaration of system-wide resources is done by annotating **two** `structs` within the `#[app]` module with the attribute `#[local]` and `#[shared]`. Each field in these structures corresponds to a different resource (identified by field name). The difference between these two sets of resources will be covered below.

Each task must declare the resources it intends to access in its corresponding metadata attribute using the `local` and `shared` arguments. Each argument takes a list of resource identifiers. The listed resources are made available to the context under the `local` and `shared` fields of the `Context` structure.

The `init` task returns the initial values for the system-wide (`#[shared]` and `#[local]`) resources.

3.5.1. `#[local]` resources

`#[local]` resources are locally accessible to a specific task, meaning that only that task can access the resource and does so without locks or critical sections. This allows for the resources, commonly drivers or large objects, to be initialized in `#[init]` and then be passed to a specific task.

Thus, a task `#[local]` resource can only be accessed by one singular task. Attempting to assign the same `#[local]` resource to more than one task is a compile-time error.

Types of `#[local]` resources must implement a `Send` trait as they are being sent from `init` to a target task, crossing a thread boundary.

The example application shown below contains three tasks `foo`, `bar` and `idle`, each having access to its own `#[local]` resource.

```rust
//! examples/locals.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965, dispatchers = [UART0, UART1])]
mod app {
    use cortex_m_semihosting::{debug, hprintln};

    #[shared]
    struct Shared {}

    #[local]
    struct Local {
        local_to_foo: i64,
        local_to_bar: i64,
        local_to_idle: i64,
    }

    // `#[init]` cannot access locals from the `#[local]` struct as they are initialized here.
    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        foo::spawn().unwrap();
        bar::spawn().unwrap();

        (
            Shared {},
            // initial values for the `#[local]` resources
            Local {
                local_to_foo: 0,
                local_to_bar: 0,
                local_to_idle: 0,
            },
        )
    }

    // `local_to_idle` can only be accessed from this context
    #[idle(local = [local_to_idle])]
```

```
44      fn idle(cx: idle::Context) -> ! {
45          let local_to_idle = cx.local.local_to_idle;
46          *local_to_idle += 1;
47
48          hprintln!("idle: local_to_idle = {}", local_to_idle);
49
50          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
51
52          // error: no `local_to_foo` field in `idle::LocalResources`
53          // _cx.local.local_to_foo += 1;
54
55          // error: no `local_to_bar` field in `idle::LocalResources`
56          // _cx.local.local_to_bar += 1;
57
58          loop {
59              cortex_m::asm::nop();
60          }
61      }
62
63      // `local_to_foo` can only be accessed from this context
64      #[task(local = [local_to_foo], priority = 1)]
65      async fn foo(cx: foo::Context) {
66          let local_to_foo = cx.local.local_to_foo;
67          *local_to_foo += 1;
68
69          // error: no `local_to_bar` field in `foo::LocalResources`
70          // cx.local.local_to_bar += 1;
71
72          hprintln!("foo: local_to_foo = {}", local_to_foo);
73      }
74
75      // `local_to_bar` can only be accessed from this context
76      #[task(local = [local_to_bar], priority = 1)]
77      async fn bar(cx: bar::Context) {
78          let local_to_bar = cx.local.local_to_bar;
79          *local_to_bar += 1;
80
81          // error: no `local_to_foo` field in `bar::LocalResources`
82          // cx.local.local_to_foo += 1;
83
84          hprintln!("bar: local_to_bar = {}", local_to_bar);
85      }
86  }
```

Running the example:

```
$ cargo xtask qemu --verbose --example locals
```
Shell

```
bar: local_to_bar = 1
foo: local_to_foo = 1
idle: local_to_idle = 1
```

Local resources in `#[init]` and `#[idle]` have `'static` lifetimes. This is safe since both tasks are not re-entrant.

3.5.1.1. Task local initialized resources

Local resources can also be specified directly in the resource claim like so: `#[task(local = [my_var: TYPE = INITIAL_VALUE, ...])]`; this allows for creating locals which do no need to be initialized in `#[init]`.

Types of `#[task(local = [..])]` resources have to be neither `Send` nor `Sync` as they are not crossing any thread boundary.

In the example below the different uses and lifetimes are shown:

```rust
//! examples/declared_locals.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965)]
mod app {
    use cortex_m_semihosting::debug;

    #[shared]
    struct Shared {}

    #[local]
    struct Local {}

    #[init(local = [a: u32 = 0])]
    fn init(cx: init::Context) -> (Shared, Local) {
        // Locals in `#[init]` have 'static lifetime
        let _a: &'static mut u32 = cx.local.a;

        debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator

        (Shared {}, Local {})
    }

    #[idle(local = [a: u32 = 0])]
```

```
32      fn idle(cx: idle::Context) -> ! {
33          // Locals in `#[idle]` have 'static lifetime
34          let _a: &'static mut u32 = cx.local.a;
35
36          loop {}
37      }
38
39      #[task(binds = UART0, local = [a: u32 = 0])]
40      fn foo(cx: foo::Context) {
41          // Locals in `#[task]`s have a local lifetime
42          let _a: &mut u32 = cx.local.a;
43
44          // error: explicit lifetime required in the type of `cx`
45          // let _a: &'static mut u32 = cx.local.a;
46      }
47  }
```

You can run the application, but as the example is designed merely to showcase the lifetime properties there is no output (it suffices to build the application).

```
$ cargo build --target thumbv7m-none-eabi --example declared_locals
```
Shell

3.5.2. `#[shared]` resources and `lock`

Critical sections are required to access `#[shared]` resources in a data race-free manner and to achieve this the `shared` field of the passed `Context` implements the `Mutex` trait for each shared resource accessible to the task. This trait has only one method, `lock`, which runs its closure argument in a critical section.

The critical section created by the `lock` API is based on dynamic priorities: it temporarily raises the dynamic priority of the context to a *ceiling* priority that prevents other tasks from preempting the critical section. This synchronization protocol is known as the Immediate Ceiling Priority Protocol (ICPP), and complies with Stack Resource Policy (SRP) based scheduling of RTIC.

In the example below we have three interrupt handlers with priorities ranging from one to three. The two handlers with the lower priorities contend for a `shared` resource and need to succeed in locking the resource in order to access its data. The highest priority handler, which does not access the `shared` resource, is free to preempt a critical section created by the lowest priority handler.

```
1   //! examples/lock.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965, dispatchers = [GPIOA, GPIOB, GPIOC])]
```
Rust

```
12 mod app {
13     use cortex_m_semihosting::{debug, hprintln};
14 
15     #[shared]
16     struct Shared {
17         shared: u32,
18     }
19 
20     #[local]
21     struct Local {}
22 
23     #[init]
24     fn init(_: init::Context) -> (Shared, Local) {
25         foo::spawn().unwrap();
26 
27         (Shared { shared: 0 }, Local {})
28     }
29 
30     // when omitted priority is assumed to be `1`
31     #[task(shared = [shared])]
32     async fn foo(mut c: foo::Context) {
33         hprintln!("A");
34 
35         // the lower priority task requires a critical section to access the
           data
36         c.shared.shared.lock(|shared| {
37             // data can only be modified within this critical section (closure)
38             *shared += 1;
39 
40             // bar will *not* run right now due to the critical section
41             bar::spawn().unwrap();
42 
43             hprintln!("B - shared = {}", *shared);
44 
45             // baz does not contend for `shared` so it's allowed to run now
46             baz::spawn().unwrap();
47         });
48 
49         // critical section is over: bar can now start
50 
51         hprintln!("E");
52 
53         debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
54     }
55 
56     #[task(priority = 2, shared = [shared])]
```

```
57      async fn bar(mut c: bar::Context) {
58          // the higher priority task does still need a critical section
59          let shared = c.shared.shared.lock(|shared| {
60              *shared += 1;
61
62              *shared
63          });
64
65          hprintln!("D - shared = {}", shared);
66      }
67
68      #[task(priority = 3)]
69      async fn baz(_: baz::Context) {
70          hprintln!("C");
71      }
72  }
```

```
$ cargo xtask qemu --verbose --example lock
```
Shell

```
A
B - shared = 1
C
D - shared = 2
E
```
Plain Text

Types of `#[shared]` resources have to be Send.

3.5.3. Multi-lock

As an extension to `lock`, and to reduce rightward drift, locks can be taken as tuples. The following examples show this in use:

```
1   //! examples/mutlilock.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965, dispatchers = [GPIOA])]
12  mod app {
13      use cortex_m_semihosting::{debug, hprintln};
14
15      #[shared]
16      struct Shared {
17          shared1: u32,
```
Rust

```
        shared2: u32,
        shared3: u32,
    }

    #[local]
    struct Local {}

    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        locks::spawn().unwrap();

        (
            Shared {
                shared1: 0,
                shared2: 0,
                shared3: 0,
            },
            Local {},
        )
    }

    // when omitted priority is assumed to be `1`
    #[task(shared = [shared1, shared2, shared3])]
    async fn locks(c: locks::Context) {
        let s1 = c.shared.shared1;
        let s2 = c.shared.shared2;
        let s3 = c.shared.shared3;

        (s1, s2, s3).lock(|s1, s2, s3| {
            *s1 += 1;
            *s2 += 1;
            *s3 += 1;

            hprintln!("Multiple locks, s1: {}, s2: {}, s3: {}", *s1, *s2, *s3);
        });

        debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
    }
}
```

```
$ cargo xtask qemu --verbose --example multilock
```
Shell

```
Multiple locks, s1: 1, s2: 1, s3: 1
```
Plain Text

3.5.4. Only shared (&-) access

By default, the framework assumes that all tasks require exclusive mutable access (`&mut-`) to resources, but it is possible to specify that a task only requires shared access (`&-`) to a resource using the `&resource_name` syntax in the `shared` list.

The advantage of specifying shared access (`&-`) to a resource is that no locks are required to access the resource even if the resource is contended by more than one task running at different priorities. The downside is that the task only gets a shared reference (`&-`) to the resource, limiting the operations it can perform on it, but where a shared reference is enough this approach reduces the number of required locks. In addition to simple immutable data, this shared access can be useful where the resource type safely implements interior mutability, with appropriate locking or atomic operations of its own.

Note that in this release of RTIC it is not possible to request both exclusive access (`&mut-`) and shared access (`&-`) to the *same* resource from different tasks. Attempting to do so will result in a compile error.

In the example below a key (e.g. a cryptographic key) is loaded (or created) at runtime (returned by `init`) and then used from two tasks that run at different priorities without any kind of lock.

Rust

```rust
//! examples/only-shared-access.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965, dispatchers = [UART0, UART1])]
mod app {
    use cortex_m_semihosting::{debug, hprintln};

    #[shared]
    struct Shared {
        key: u32,
    }

    #[local]
    struct Local {}

    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        foo::spawn().unwrap();
        bar::spawn().unwrap();

        (Shared { key: 0xdeadbeef }, Local {})
    }

    #[task(shared = [&key])]
```

```
32      async fn foo(cx: foo::Context) {
33          let key: &u32 = cx.shared.key;
34          hprintln!("foo(key = {:#x})", key);
35
36          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
37      }
38
39      #[task(priority = 2, shared = [&key])]
40      async fn bar(cx: bar::Context) {
41          hprintln!("bar(key = {:#x})", cx.shared.key);
42      }
43  }
```

```
$ cargo xtask qemu --verbose --example only-shared-access
```
Shell

```
bar(key = 0xdeadbeef)
foo(key = 0xdeadbeef)
```
Plain Text

3.5.5. Lock-free access of shared resources

A critical section is *not* required to access a `#[shared]` resource that's only accessed by tasks running at the *same* priority. In this case, you can opt out of the `lock` API by adding the `#[lock_free]` field-level attribute to the resource declaration (see example below).

To adhere to the Rust aliasing rule, a resource may be either accessed through multiple immutable references or a singe mutable reference (but not both at the same time).

Using `#[lock_free]` on resources shared by tasks running at different priorities will result in a *compile-time* error – not using the `lock` API would violate the aforementioned alias rule. Similarly, for each priority there can be only a single *software* task accessing a shared resource (as an `async` task may yield execution to other *software* or *hardware* tasks running at the same priority). However, under this single-task restriction, we make the observation that the resource is in effect no longer `shared` but rather `local`. Thus, using a `#[lock_free]` shared resource will result in a *compile-time* error – where applicable, use a `#[local]` resource instead.

```
1   //! examples/lock-free.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965)]
12  mod app {
13      use cortex_m_semihosting::{debug, hprintln};
14      use lm3s6965::Interrupt;
15
```
Rust

```
16      #[shared]
17      struct Shared {
18          #[lock_free] // <- lock-free shared resource
19          counter: u64,
20      }
21
22      #[local]
23      struct Local {}
24
25      #[init]
26      fn init(_: init::Context) -> (Shared, Local) {
27          rtic::pend(Interrupt::UART0);
28
29          (Shared { counter: 0 }, Local {})
30      }
31
32      #[task(binds = UART0, shared = [counter])] // <- same priority
33      fn foo(c: foo::Context) {
34          rtic::pend(Interrupt::UART1);
35
36          *c.shared.counter += 1; // <- no lock API required
37          let counter = *c.shared.counter;
38          hprintln!("  foo = {}", counter);
39      }
40
41      #[task(binds = UART1, shared = [counter])] // <- same priority
42      fn bar(c: bar::Context) {
43          rtic::pend(Interrupt::UART0);
44          *c.shared.counter += 1; // <- no lock API required
45          let counter = *c.shared.counter;
46          hprintln!("  bar = {}", counter);
47
48          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
49      }
50  }
```

```
$ cargo xtask qemu --verbose --example lock-free
```
Shell

```
  foo = 1
  bar = 2
```
Plain Text

3.6. App initialization and the `#[init]` task

An RTIC application requires an `init` task setting up the system. The corresponding `init` function must have the signature `fn(init::Context) -> (Shared, Local)`, where `Shared` and `Local` are resource structures defined by the user.

The `init` task executes after system reset, [after an optionally defined `pre-init` code section] and an always occurring internal RTIC initialization.

The `init` and optional `pre-init` tasks runs *with interrupts disabled* and have exclusive access to Cortex-M (the `critical_section::CriticalSection` token is available as `cs`).

Device specific peripherals are available through the `core` and `device` fields of `init::Context`.

3.6.1. Example

The example below shows the types of the `core`, `device` and `cs` fields, and showcases the use of a `local` variable with `'static` lifetime. Such variables can be delegated from the `init` task to other tasks of the RTIC application.

The `device` field is only available when the `peripherals` argument is set to the default value `true`. In the rare case you want to implement an ultra-slim application you can explicitly set `peripherals` to `false`.

```rust
//! examples/init.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965, peripherals = true)]
mod app {
    use cortex_m_semihosting::{debug, hprintln};

    #[shared]
    struct Shared {}

    #[local]
    struct Local {}

    #[init(local = [x: u32 = 0])]
    fn init(cx: init::Context) -> (Shared, Local) {
        // Cortex-M peripherals
        let _core: cortex_m::Peripherals = cx.core;

        // Device specific peripherals
        let _device: lm3s6965::Peripherals = cx.device;

        // Locals in `init` have 'static lifetime
        let _x: &'static mut u32 = cx.local.x;

        // Access to the critical section token,
        // to indicate that this is a critical section
        let _cs_token: rtic::export::CriticalSection = cx.cs;
```

```
35
36          hprintln!("init");
37
38          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
39
40          (Shared {}, Local {})
41      }
42  }
```

Running the example will print `init` to the console and then exit the QEMU process.

```
$ cargo xtask qemu --verbose --example init
```
Shell

```
init
```
Plain Text

3.7. The background task `#[idle]`

A function marked with the `idle` attribute can optionally appear in the module. This becomes the special *idle task* and must have signature `fn(idle::Context) -> !`.

When present, the runtime will execute the `idle` task after `init`. Unlike `init`, `idle` will run *with interrupts enabled* and must never return, as the `-> !` function signature indicates. The Rust type `!` means "never".

Like in `init`, locally declared resources will have `'static` lifetimes that are safe to access.

The example below shows that `idle` runs after `init`.

```rust
1   //! examples/idle.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965)]
12  mod app {
13      use cortex_m_semihosting::{debug, hprintln};
14
15      #[shared]
16      struct Shared {}
17
18      #[local]
19      struct Local {}
20
21      #[init]
22      fn init(_: init::Context) -> (Shared, Local) {
```
Rust

```
        hprintln!("init");

        (Shared {}, Local {})
    }

    #[idle(local = [x: u32 = 0])]
    fn idle(cx: idle::Context) -> ! {
        // Locals in idle have lifetime 'static
        let _x: &'static mut u32 = cx.local.x;

        hprintln!("idle");

        debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator

        loop {
            cortex_m::asm::nop();
        }
    }
}
```

```
$ cargo xtask qemu --verbose --example idle
```
Shell

```
init
idle
```
Plain Text

By default, the RTIC `idle` task does not try to optimize for any specific targets.

A common useful optimization is to enable the SLEEPONEXIT and allow the MCU to enter sleep when reaching `idle`.

> **Caution**: some hardware unless configured disables the debug unit during sleep mode.
>
> Consult your hardware specific documentation as this is outside the scope of RTIC.

The following example shows how to enable sleep by setting the `SLEEPONEXIT` and providing a custom `idle` task replacing the default `nop()` with `wfi()`.

```rust
//! examples/idle-wfi.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965)]
mod app {
```
Rust

```
13      use cortex_m_semihosting::{debug, hprintln};
14
15      #[shared]
16      struct Shared {}
17
18      #[local]
19      struct Local {}
20
21      #[init]
22      fn init(mut cx: init::Context) -> (Shared, Local) {
23          hprintln!("init");
24
25          // Set the ARM SLEEPONEXIT bit to go to sleep after handling interrupts
26          // See https://developer.arm.com/documentation/100737/0100/Power-management/Sleep-mode/Sleep-on-exit-bit
27          cx.core.SCB.set_sleeponexit();
28
29          (Shared {}, Local {})
30      }
31
32      #[idle(local = [x: u32 = 0])]
33      fn idle(cx: idle::Context) -> ! {
34          // Locals in idle have lifetime 'static
35          let _x: &'static mut u32 = cx.local.x;
36
37          hprintln!("idle");
38
39          debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
40
41          loop {
42              // Now Wait For Interrupt is used instead of a busy-wait loop
43              // to allow MCU to sleep between interrupts
44              // https://developer.arm.com/documentation/ddi0406/c/Application-Level-Architecture/Instruction-Details/Alphabetical-list-of-instructions/WFI
45              rtic::export::wfi()
46          }
47      }
48  }
```

```
$ cargo xtask qemu --verbose --example idle-wfi
```
Shell

```
init
idle
```
Plain Text

Notice: The `idle` task cannot be used together with *software* tasks running at priority zero. The reason is that `idle` is running as a non-returning Rust function at priority zero. Thus there would be no way for an executor at priority zero to give control to *software* tasks at the same priority.

3.8. Communication over channels.

Channels can be used to communicate data between running tasks. The channel is essentially a wait queue, allowing tasks with multiple producers and a single receiver. A channel is constructed in the `init` task and backed by statically allocated memory. Send and receive endpoints are distributed to *software* tasks:

```Rust
...
const CAPACITY: usize = 5;
#[init]
    fn init(_: init::Context) -> (Shared, Local) {
        let (s, r) = make_channel!(u32, CAPACITY);
        receiver::spawn(r).unwrap();
        sender1::spawn(s.clone()).unwrap();
        sender2::spawn(s.clone()).unwrap();
        ...
```

In this case the channel holds data of `u32` type with a capacity of 5 elements.

Channels can also be used from *hardware* tasks, but only in a non-`async` manner using the Try API.

3.8.1. Sending data

The `send` method post a message on the channel as shown below:

```Rust
#[task]
async fn sender1(_c: sender1::Context, mut sender: Sender<'static, u32, CAPACITY>) {
    hprintln!("Sender 1 sending: 1");
    sender.send(1).await.unwrap();
}
```

3.8.2. Receiving data

The receiver can `await` incoming messages:

```Rust
#[task]
async fn receiver(_c: receiver::Context, mut receiver: Receiver<'static, u32,
CAPACITY>) {
    while let Ok(val) = receiver.recv().await {
        hprintln!("Receiver got: {}", val);
        ...
    }
}
```

Channels are implemented using a small (global) *Critical Section* (CS) for protection against race-conditions. The user must provide an CS implementation. For the examples a CS implementation is provided by a platform crate, for example `cortex-m/critical-section-single-core` or `esp32c3/critical-section`.

For a complete example:

Rust

```rust
//! examples/async-channel.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965, dispatchers = [SSI0])]
mod app {
    use cortex_m_semihosting::{debug, hprintln};
    use rtic_sync::{channel::*, make_channel};

    #[shared]
    struct Shared {}

    #[local]
    struct Local {}

    const CAPACITY: usize = 5;
    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        let (s, r) = make_channel!(u32, CAPACITY);

        receiver::spawn(r).unwrap();
        sender1::spawn(s.clone()).unwrap();
        sender2::spawn(s.clone()).unwrap();
        sender3::spawn(s).unwrap();

        (Shared {}, Local {})
    }

    #[task]
    async fn receiver(_c: receiver::Context, mut receiver: Receiver<'static, u32, CAPACITY>) {
        while let Ok(val) = receiver.recv().await {
            hprintln!("Receiver got: {}", val);
            if val == 3 {
                debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
            }
        }
    }

    #[task]
```

```
46      async fn sender1(_c: sender1::Context, mut sender: Sender<'static, u32,
        CAPACITY>) {
47          hprintln!("Sender 1 sending: 1");
48          sender.send(1).await.unwrap();
49      }
50
51      #[task]
52      async fn sender2(_c: sender2::Context, mut sender: Sender<'static, u32,
        CAPACITY>) {
53          hprintln!("Sender 2 sending: 2");
54          sender.send(2).await.unwrap();
55      }
56
57      #[task]
58      async fn sender3(_c: sender3::Context, mut sender: Sender<'static, u32,
        CAPACITY>) {
59          hprintln!("Sender 3 sending: 3");
60          sender.send(3).await.unwrap();
61      }
62  }
```

```
$ cargo xtask qemu --verbose --example async-channel
```
Shell

```
Sender 1 sending: 1
Sender 2 sending: 2
Sender 3 sending: 3
Receiver got: 1
Receiver got: 2
Receiver got: 3
```
Plain Text

Also sender endpoint can be awaited. In case the channel capacity has not yet been reached, `await`-ing the sender can progress immediately, while in the case the capacity is reached, the sender is blocked until there is free space in the queue. In this way data is never lost.

In the following example the `CAPACITY` has been reduced to 1, forcing sender tasks to wait until the data in the channel has been received.

```
1   //! examples/async-channel-done.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965, dispatchers = [SSI0])]
```
Rust

```
12 mod app {
13     use cortex_m_semihosting::{debug, hprintln};
14     use rtic_sync::{channel::*, make_channel};
15 
16     #[shared]
17     struct Shared {}
18 
19     #[local]
20     struct Local {}
21 
22     const CAPACITY: usize = 1;
23     #[init]
24     fn init(_: init::Context) -> (Shared, Local) {
25         let (s, r) = make_channel!(u32, CAPACITY);
26 
27         receiver::spawn(r).unwrap();
28         sender1::spawn(s.clone()).unwrap();
29         sender2::spawn(s.clone()).unwrap();
30         sender3::spawn(s).unwrap();
31 
32         (Shared {}, Local {})
33     }
34 
35     #[task]
36     async fn receiver(_c: receiver::Context, mut receiver: Receiver<'static,
    u32, CAPACITY>) {
37         while let Ok(val) = receiver.recv().await {
38             hprintln!("Receiver got: {}", val);
39             if val == 3 {
40                 debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
41             }
42         }
43     }
44 
45     #[task]
46     async fn sender1(_c: sender1::Context, mut sender: Sender<'static, u32,
    CAPACITY>) {
47         hprintln!("Sender 1 sending: 1");
48         sender.send(1).await.unwrap();
49         hprintln!("Sender 1 done");
50     }
51 
52     #[task]
53     async fn sender2(_c: sender2::Context, mut sender: Sender<'static, u32,
    CAPACITY>) {
54         hprintln!("Sender 2 sending: 2");
```

```
55          sender.send(2).await.unwrap();
56          hprintln!("Sender 2 done");
57      }
58
59      #[task]
60      async fn sender3(_c: sender3::Context, mut sender: Sender<'static, u32,
        CAPACITY>) {
61          hprintln!("Sender 3 sending: 3");
62          sender.send(3).await.unwrap();
63          hprintln!("Sender 3 done");
64      }
65  }
```

Looking at the output, we find that `Sender 2` will wait until the data sent by `Sender 1` as been received.

> **NOTICE** *Software* tasks at the same priority are executed asynchronously to each other, thus **NO** strict order can be assumed. (The presented order here applies only to the current implementation, and may change between RTIC framework releases.)

```
$ cargo xtask qemu --verbose --example async-channel-done"
```
Shell

```
Sender 1 sending: 1
Sender 1 done
Sender 2 sending: 2
Sender 3 sending: 3
Receiver got: 1
Sender 2 done
Receiver got: 2
Sender 3 done
Receiver got: 3
```
Plain Text

3.8.3. Error handling

In case all senders have been dropped `await`-ing on an empty receiver channel results in an error. This allows to gracefully implement different types of shutdown operations.

```
1   //! examples/async-channel-no-sender.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965, dispatchers = [SSI0])]
12  mod app {
```
Rust

```
    use cortex_m_semihosting::{debug, hprintln};
    use rtic_sync::{channel::*, make_channel};

    #[shared]
    struct Shared {}

    #[local]
    struct Local {}

    const CAPACITY: usize = 1;
    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        let (_s, r) = make_channel!(u32, CAPACITY);

        receiver::spawn(r).unwrap();

        (Shared {}, Local {})
    }

    #[task]
    async fn receiver(_c: receiver::Context, mut receiver: Receiver<'static,
    u32, CAPACITY>) {
        hprintln!("Receiver got: {:?}", receiver.recv().await);

        debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
    }
}
```

```
$ cargo xtask qemu --verbose --example async-channel-no-sender
```
Shell

```
Receiver got: Err(NoSender)
```
Plain Text

Similarly, `await`-ing on a send channel results in an error in case the receiver has been dropped. This allows to gracefully implement application level error handling.

The resulting error returns the data back to the sender, allowing the sender to take appropriate action (e.g., storing the data to later retry sending it).

```
//! examples/async-channel-no-receiver.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

```
Rust

```rust
#[rtic::app(device = lm3s6965, dispatchers = [SSI0])]
mod app {
    use cortex_m_semihosting::{debug, hprintln};
    use rtic_sync::{channel::*, make_channel};

    #[shared]
    struct Shared {}

    #[local]
    struct Local {}

    const CAPACITY: usize = 1;
    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        let (s, _r) = make_channel!(u32, CAPACITY);

        sender1::spawn(s.clone()).unwrap();

        (Shared {}, Local {})
    }

    #[task]
    async fn sender1(_c: sender1::Context, mut sender: Sender<'static, u32,
    CAPACITY>) {
        hprintln!("Sender 1 sending: 1 {:?}", sender.send(1).await);
        debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
    }
}
```

```
$ cargo xtask qemu --verbose --example async-channel-no-receiver
```
Shell

```
Sender 1 sending: 1 Err(NoReceiver(1))
```
Plain Text

3.8.4. Try API

Using the Try API, you can send or receive data from or to a channel without requiring that the operation succeeds, and in non-`async` contexts.

This API is exposed through `Receiver::try_recv` and `Sender::try_send`.

```rust
//! examples/async-channel-try.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;
```
Rust

```
10
11 #[rtic::app(device = lm3s6965, dispatchers = [SSI0])]
12 mod app {
13     use cortex_m_semihosting::{debug, hprintln};
14     use rtic_sync::{channel::*, make_channel};
15
16     #[shared]
17     struct Shared {}
18
19     #[local]
20     struct Local {
21         sender: Sender<'static, u32, CAPACITY>,
22     }
23
24     const CAPACITY: usize = 1;
25     #[init]
26     fn init(_: init::Context) -> (Shared, Local) {
27         let (s, r) = make_channel!(u32, CAPACITY);
28
29         receiver::spawn(r).unwrap();
30         sender1::spawn(s.clone()).unwrap();
31
32         (Shared {}, Local { sender: s.clone() })
33     }
34
35     #[task]
36     async fn receiver(_c: receiver::Context, mut receiver: Receiver<'static,
    u32, CAPACITY>) {
37         while let Ok(val) = receiver.recv().await {
38             hprintln!("Receiver got: {}", val);
39         }
40     }
41
42     #[task]
43     async fn sender1(_c: sender1::Context, mut sender: Sender<'static, u32,
    CAPACITY>) {
44         hprintln!("Sender 1 sending: 1");
45         sender.send(1).await.unwrap();
46         hprintln!("Sender 1 try sending: 2 {:?}", sender.try_send(2));
47         debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
48     }
49
50     // This interrupt is never triggered, but is used to demonstrate that
51     // one can (try to) send data into a channel from a hardware task.
52     #[task(binds = GPIOA, local = [sender])]
53     fn hw_task(cx: hw_task::Context) {
```

```
54          cx.local.sender.try_send(3).ok();
55      }
56  }
```

```
$ cargo xtask qemu --verbose --example async-channel-try
```
Shell

```
Sender 1 sending: 1
Sender 1 try sending: 2 Err(Full(2))
```
Plain Text

3.9. Delay and Timeout using Monotonics

A convenient way to express miniminal timing requirements is by delaying progression.

This can be achieved by instantiating a monotonic timer (for implementations, see `rtic-monotonics`). Monotonics can be thought of like timekeepers, they measure the amount of time elapsed since the start of the monotonic (usually the start of the application). In RTIC, monotonics can be used for delaying the execution of code and for time measurement.

All monotonic implementations in RTIC are guaranteed to remain stable longer than the lifetime of the hardware, i.e. you can theoretically delay a task for multiple years and it should execute successfully (presuming there's no hardware failure). The resolution of the monotonic (i.e. the smallest possible delay that you can sleep) depends on the implementation. Many monotonic implementations also allow you to control the resolution by setting a prescaler for the hardware timer. This is documented in the crate docs of the individual `rtic-monotonics`.

3.9.1. Delay

```rust
25      #[init]
26      fn init(cx: init::Context) -> (Shared, Local) {
27          hprintln!("init");
28
29          Mono::start(cx.core.SYST, 12_000_000);
```
Rust

A *software* task can `await` the delay to expire:

```rust
#[task]
async fn foo(_cx: foo::Context) {
    ...
    Mono::delay(100.millis()).await;
    ...
}
```
Rust

A complete example

```rust
1   //! examples/async-delay.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
```
Rust

```
9  use panic_semihosting as _;
10
11 #[rtic::app(device = lm3s6965, dispatchers = [SSI0, UART0], peripherals = true)]
12 mod app {
13     use cortex_m_semihosting::{debug, hprintln};
14     use rtic_monotonics::systick::prelude::*;
15
16     systick_monotonic!(Mono, 100);
17
18     #[shared]
19     struct Shared {}
20
21     #[local]
22     struct Local {}
23
24     #[init]
25     fn init(cx: init::Context) -> (Shared, Local) {
26         hprintln!("init");
27
28         Mono::start(cx.core.SYST, 12_000_000);
29
30         foo::spawn().ok();
31         bar::spawn().ok();
32         baz::spawn().ok();
33
34         (Shared {}, Local {})
35     }
36
37     #[task]
38     async fn foo(_cx: foo::Context) {
39         hprintln!("hello from foo");
40         Mono::delay(100.millis()).await;
41         hprintln!("bye from foo");
42     }
43
44     #[task]
45     async fn bar(_cx: bar::Context) {
46         hprintln!("hello from bar");
47         Mono::delay(200.millis()).await;
48         hprintln!("bye from bar");
49     }
50
51     #[task]
52     async fn baz(_cx: baz::Context) {
53         hprintln!("hello from baz");
54         Mono::delay(300.millis()).await;
```

```
55          hprintln!("bye from baz");
56
57          debug::exit(debug::EXIT_SUCCESS);
58      }
59  }
```

```
$ cargo xtask qemu --verbose --example async-delay
```
Shell

```
init
hello from bar
hello from baz
hello from foo
bye from foo
bye from bar
bye from baz
```
Plain Text

> Interested in contributing new implementations of `Monotonic`, or more information about the inner workings of monotonics? Check out the Implementing a `Monotonic` chapter!

3.9.2. Timeout

Rust `Future`s (underlying Rust `async`/`await`) are composable. This makes it possible to `select` in between `Futures` that have completed.

A common use case is transactions with an associated timeout. In the examples shown below, we introduce a fake HAL device that performs some imagined transaction when you call `hal_get(n).await`. We have modelled the time it takes based on the input parameter (`n`) as `350ms + n * 100ms`.

Using the `select_biased` macro from the `futures` crate it may look like this:

```
40          // Call hal with short relative timeout using `select_biased`
41          select_biased! {
42              v = hal_get(1).fuse() => hprintln!("hal returned {}", v),
43              _ = Mono::delay(200.millis()).fuse() =>  hprintln!("timeout", ), //
                this will finish first
44          }
45
46          // Call hal with long relative timeout using `select_biased`
47          select_biased! {
48              v = hal_get(1).fuse() => hprintln!("hal returned {}", v), // hal
                finish first
49              _ = Mono::delay(1000.millis()).fuse() =>  hprintln!("timeout", ),
50          }
```
Rust

Assuming the `hal_get` will take 450ms to finish, a short timeout of 200ms will expire before `hal_get` can complete.

Extending the timeout to 1000ms would cause `hal_get` will to complete first.

Using `select_biased` any number of futures can be combined, so its very powerful. However, as the timeout pattern is frequently used, more ergonomic support is baked into RTIC, provided by

the `rtic-monotonics` and `rtic-time` crates. Here's another example, using `Mono::delay_until` and `Mono::timeout_after`:

```Rust
        // get the current time instance
        let mut instant = Mono::now();

        // do this 3 times
        for n in 0..3 {
            // absolute point in time without drift
            instant += 1000.millis();
            Mono::delay_until(instant).await;

            // absolute point in time for timeout
            let timeout = instant + 500.millis();
            hprintln!("now is {:?}, timeout at {:?}", Mono::now(), timeout);

            match Mono::timeout_at(timeout, hal_get(n)).await {
                Ok(v) => hprintln!("hal returned {} at time {:?}", v,
                Mono::now()),
                _ => hprintln!("timeout"),
            }
        }
```

In cases where you want exact control over time without drift we can use exact points in time using `Instant`, and spans of time using `Duration`. Operations on the `Instant` and `Duration` types come from the `fugit` crate.

`let mut instant = Mono::now()` sets the starting time of execution.

We want to call `hal_get` every 1000ms relative to this starting time. We accomplish this by incrementing our `instant` by 1000 ms and then using `Mono::delay_until(instant).await`. Any additional delays incurred as we iterate around this loop are compensated for by delaying until 'previous + 1000' as opposed to 'now + 1000' (which would cause our loop timing to drift).

To show an alternative to the `select!` async timeout example above, we define a future point in time as `timeout`, and call `Mono::timeout_at(timeout, hal_get(n)).await`.

For the first iteration of the loop, with `n == 0`, the `hal_get` will take 350ms (as described above), and finishes before the timeout. For the second iteration, the delay is 450ms, which still finishes before the timeout. For the third iteration, with `n == 2`, `hal_get` will take 550ms to finish, in which case we will run into a timeout.

A complete example

```Rust
//! examples/async-timeout.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]
```

```
8
9   use cortex_m_semihosting::{debug, hprintln};
10  use panic_semihosting as _;
11  use rtic_monotonics::systick::prelude::*;
12  systick_monotonic!(Mono, 100);
13
14  #[rtic::app(device = lm3s6965, dispatchers = [SSI0, UART0], peripherals = true)]
15  mod app {
16      use super::*;
17      use futures::{future::FutureExt, select_biased};
18
19      #[shared]
20      struct Shared {}
21
22      #[local]
23      struct Local {}
24
25      // ANCHOR: init
26      #[init]
27      fn init(cx: init::Context) -> (Shared, Local) {
28          hprintln!("init");
29
30          Mono::start(cx.core.SYST, 12_000_000);
31          // ANCHOR_END: init
32
33          foo::spawn().ok();
34
35          (Shared {}, Local {})
36      }
37
38      #[task]
39      async fn foo(_cx: foo::Context) {
40          // ANCHOR: select_biased
41          // Call hal with short relative timeout using `select_biased`
42          select_biased! {
43              v = hal_get(1).fuse() => hprintln!("hal returned {}", v),
44              _ = Mono::delay(200.millis()).fuse() =>  hprintln!("timeout", ), //
                this will finish first
45          }
46
47          // Call hal with long relative timeout using `select_biased`
48          select_biased! {
49              v = hal_get(1).fuse() => hprintln!("hal returned {}", v), // hal
                finish first
50              _ = Mono::delay(1000.millis()).fuse() =>  hprintln!("timeout", ),
51          }
```

```
        // ANCHOR_END: select_biased

        // ANCHOR: timeout_after_basic
        // Call hal with long relative timeout using monotonic `timeout_after`
        match Mono::timeout_after(1000.millis(), hal_get(1)).await {
            Ok(v) => hprintln!("hal returned {}", v),
            _ => hprintln!("timeout"),
        }
        // ANCHOR_END: timeout_after_basic

        // ANCHOR: timeout_at_basic
        // get the current time instance
        let mut instant = Mono::now();

        // do this 3 times
        for n in 0..3 {
            // absolute point in time without drift
            instant += 1000.millis();
            Mono::delay_until(instant).await;

            // absolute point in time for timeout
            let timeout = instant + 500.millis();
            hprintln!("now is {:?}, timeout at {:?}", Mono::now(), timeout);

            match Mono::timeout_at(timeout, hal_get(n)).await {
                Ok(v) => hprintln!("hal returned {} at time {:?}", v,
                Mono::now()),
                _ => hprintln!("timeout"),
            }
        }
        // ANCHOR_END: timeout_at_basic

        debug::exit(debug::EXIT_SUCCESS);
    }
}

// Emulate some hal
async fn hal_get(n: u32) -> u32 {
    // emulate some delay time dependent on n
    let d = 350.millis() + n * 100.millis();
    hprintln!("the hal takes a duration of {:?}", d);
    Mono::delay(d).await;
    // emulate some return value
    5
}
```

```
$ cargo xtask qemu --verbose --example async-timeout
```
Shell

```
init
the hal takes a duration of Duration { ticks: 45 }
timeout
the hal takes a duration of Duration { ticks: 45 }
hal returned 5
the hal takes a duration of Duration { ticks: 45 }
hal returned 5
now is Instant { ticks: 213 }, timeout at Instant { ticks: 263 }
the hal takes a duration of Duration { ticks: 35 }
hal returned 5 at time Instant { ticks: 249 }
now is Instant { ticks: 313 }, timeout at Instant { ticks: 363 }
the hal takes a duration of Duration { ticks: 45 }
hal returned 5 at time Instant { ticks: 359 }
now is Instant { ticks: 413 }, timeout at Instant { ticks: 463 }
the hal takes a duration of Duration { ticks: 55 }
timeout
```
Plain Text

3.10. The minimal app

This is the smallest possible RTIC application:

```rust
//! examples/smallest.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use core::panic::PanicInfo;
use cortex_m_semihosting::debug;
use rtic::app;

#[app(device = lm3s6965)]
mod app {
    use super::*;

    #[shared]
    struct Shared {}

    #[local]
    struct Local {}

    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
```
Rust

```
        (Shared {}, Local {})
    }
}

#[panic_handler]
fn panic_handler(_: &PanicInfo) -> ! {
    debug::exit(debug::EXIT_FAILURE);
    loop {}
}
```

RTIC is designed with resource efficiency in mind. RTIC itself does not rely on any dynamic memory allocation, thus RAM requirement is dependent only on the application. The flash memory footprint is below 1kB including the interrupt vector table.

For a minimal example you can expect something like:

```
$ cargo xtask size --example smallest --backend thumbv7
```
Shell

```
   text    data     bss     dec     hex filename
    604       0       4     608     260 smallest
```
Plain Text

3.10.1. Tips & tricks

In this section we will explore common tips & tricks related to using RTIC.

3.10.2. Resource de-structure-ing

Destructuring task resources might help readability if a task takes multiple resources. Here are two examples on how to split up the resource struct:

```
//! examples/destructure.rs

#![no_main]
#![no_std]
#![deny(warnings)]
#![deny(unsafe_code)]
#![deny(missing_docs)]

use panic_semihosting as _;

#[rtic::app(device = lm3s6965, dispatchers = [UART0])]
mod app {
    use cortex_m_semihosting::{debug, hprintln};

    #[shared]
    struct Shared {
        a: u32,
        b: u32,
        c: u32,
    }

```
Rust

```
    #[local]
    struct Local {}

    #[init]
    fn init(_: init::Context) -> (Shared, Local) {
        foo::spawn().unwrap();
        bar::spawn().unwrap();

        (Shared { a: 0, b: 1, c: 2 }, Local {})
    }

    #[idle]
    fn idle(_: idle::Context) -> ! {
        debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
        loop {}
    }

    // Direct destructure
    #[task(shared = [&a, &b, &c], priority = 1)]
    async fn foo(cx: foo::Context) {
        let a = cx.shared.a;
        let b = cx.shared.b;
        let c = cx.shared.c;

        hprintln!("foo: a = {}, b = {}, c = {}", a, b, c);
    }

    // De-structure-ing syntax
    #[task(shared = [&a, &b, &c], priority = 1)]
    async fn bar(cx: bar::Context) {
        let bar::SharedResources { a, b, c, .. } = cx.shared;

        hprintln!("bar: a = {}, b = {}, c = {}", a, b, c);
    }
}
```

```
$ cargo xtask qemu --verbose --example destructure
```
Shell

```
bar: a = 0, b = 1, c = 2
foo: a = 0, b = 1, c = 2
```
Plain Text

3.10.3. Using indirection for faster message passing

Message passing always involves copying the payload from the sender into a static variable and then from the static variable into the receiver. Thus sending a large buffer, like a `[u8; 128]`, as a message involves two expensive `memcpy`s.

Indirection can minimize message passing overhead: instead of sending the buffer by value, one can send an owning pointer into the buffer.

One can use a global memory allocator to achieve indirection (`alloc::Box`, `alloc::Rc`, etc.), which requires using the nightly channel as of Rust v1.37.0, or one can use a statically allocated memory pool like `heapless::Pool`.

As this example of approach goes completely outside of RTIC resource model with shared and local the program would rely on the correctness of the memory allocator, in this case `heapless::pool`.

Here's an example where `heapless::Pool` is used to "box" buffers of 128 bytes.

```Rust
//! examples/pool.rs

#![no_main]
#![no_std]
#![deny(warnings)]

use panic_semihosting as _;
use rtic::app;

// thumbv6-none-eabi does not support pool
// This might be better worked around in the build system,
// but for proof of concept, let's try having one example
// being different for different backends
// https://docs.rs/heapless/0.8.0/heapless/pool/index.html#target-support
cfg_if::cfg_if! {
    if #[cfg(feature = "thumbv6-backend")] {
        // Copy of the smallest.rs example
        #[app(device = lm3s6965)]
        mod app {
            use cortex_m_semihosting::debug;

            #[shared]
            struct Shared {}

            #[local]
            struct Local {}

            #[init]
            fn init(_: init::Context) -> (Shared, Local) {
                debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
                (Shared {}, Local {})
            }
        }
    } else {
        // Run actual pool example
        use heapless::{
            box_pool,
            pool::boxed::{Box, BoxBlock},
        };
```

```
40
41          // Declare a pool containing 8-byte memory blocks
42          box_pool!(P: u8);
43
44          const POOL_CAPACITY: usize = 512;
45
46          #[app(device = lm3s6965, dispatchers = [SSI0, QEI0])]
47          mod app {
48              use crate::{Box, BoxBlock, POOL_CAPACITY};
49              use cortex_m_semihosting::debug;
50              use lm3s6965::Interrupt;
51
52              // Import the memory pool into scope
53              use crate::P;
54
55              #[shared]
56              struct Shared {}
57
58              #[local]
59              struct Local {}
60
61              const BLOCK: BoxBlock<u8> = BoxBlock::new();
62
63              #[init(local = [memory: [BoxBlock<u8>; POOL_CAPACITY] = [BLOCK;
                POOL_CAPACITY]])]
64              fn init(cx: init::Context) -> (Shared, Local) {
65                  for block in cx.local.memory {
66                      // Give the 'static memory to the pool
67                      P.manage(block);
68                  }
69
70                  rtic::pend(Interrupt::I2C0);
71
72                  (Shared {}, Local {})
73              }
74
75              #[task(binds = I2C0, priority = 2)]
76              fn i2c0(_: i2c0::Context) {
77                  // Claim 128 u8 blocks
78                  let x = P.alloc(128).unwrap();
79
80                  // .. send it to the `foo` task
81                  foo::spawn(x).ok().unwrap();
82
83                  // send another 128 u8 blocks to the task `bar`
84                  bar::spawn(P.alloc(128).unwrap()).ok().unwrap();
```

```
85              }
86
87              #[task]
88              async fn foo(_: foo::Context, _x: Box<P>) {
89                  // explicitly return the block to the pool
90                  drop(_x);
91
92                  debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
93              }
94
95              #[task(priority = 2)]
96              async fn bar(_: bar::Context, _x: Box<P>) {
97                  // this is done automatically so we can omit the call to `drop`
98                  // drop(_x);
99              }
100         }
101     }
102 }
```

```
$ cargo xtask qemu --verbose --example pool
```
Shell

3.10.4. ‘static super-powers

In `#[init]`, `#[idle]` and divergent software tasks `local` resources have `'static` lifetime.

Useful when pre-allocating and/or splitting resources between tasks, drivers or some other object. This comes in handy when drivers, such as USB drivers, need to allocate memory and when using splittable data structures such as `heapless::spsc::Queue`.

In the following example two different tasks share a `heapless::spsc::Queue` for lock-free access to the shared queue.

```
1   //! examples/static-resources-in-init.rs
2
3   #![no_main]
4   #![no_std]
5   #![deny(warnings)]
6   #![deny(unsafe_code)]
7   #![deny(missing_docs)]
8
9   use panic_semihosting as _;
10
11  #[rtic::app(device = lm3s6965, dispatchers = [UART0])]
12  mod app {
13      use cortex_m_semihosting::{debug, hprintln};
14      use heapless::spsc::{Consumer, Producer, Queue};
15
16      #[shared]
```
Rust

```
17      struct Shared {}
18
19      #[local]
20      struct Local {
21          p: Producer<'static, u32, 5>,
22          c: Consumer<'static, u32, 5>,
23      }
24
25      #[init(local = [q: Queue<u32, 5> = Queue::new()])]
26      fn init(cx: init::Context) -> (Shared, Local) {
27          // q has 'static life-time so after the split and return of `init`
28          // it will continue to exist and be allocated
29          let (p, c) = cx.local.q.split();
30
31          foo::spawn().unwrap();
32
33          (Shared {}, Local { p, c })
34      }
35
36      #[idle(local = [c])]
37      fn idle(c: idle::Context) -> ! {
38          loop {
39              // Lock-free access to the same underlying queue!
40              if let Some(data) = c.local.c.dequeue() {
41                  hprintln!("received message: {}", data);
42
43                  // Run foo until data
44                  if data == 3 {
45                      debug::exit(debug::EXIT_SUCCESS); // Exit QEMU simulator
46                  } else {
47                      foo::spawn().unwrap();
48                  }
49              }
50          }
51      }
52
53      #[task(local = [p, state: u32 = 0], priority = 1)]
54      async fn foo(c: foo::Context) {
55          *c.local.state += 1;
56
57          // Lock-free access to the same underlying queue!
58          c.local.p.enqueue(*c.local.state).unwrap();
59      }
60  }
```

Running this program produces the expected output.

```
$ cargo xtask qemu --verbose --example static-resources-in-init
```
Shell

```
received message: 1
received message: 2
received message: 3
```
Plain Text

3.10.5. Inspecting generated code

`#[rtic::app]` is a procedural macro that produces support code. If for some reason you need to inspect the code generated by this macro you have two options:

- You can inspect the file `rtic-expansion.rs` inside the `target` directory.
- Use the `cargo-expand` sub-command

3.10.5.1. Using generated `rtic-expansion.rs`

Locating this file depends on how building is performed.

Using e.g. `cargo xtask build-example` within the main RTIC repo will place the file based on "platform" used:

```
$ cargo xtask example-build --example smallest
$ cargo xtask example-build --example monotonic --platform esp32-c3

$ fd -u rtic-expansion.rs
examples/esp32c3/target/rtic-expansion.rs
examples/lm3s6965/target/rtic-expansion.rs
```

In the regular cargo project case it goes directly in the `target` folder.

This file contains the expansion of the `#[rtic::app]` item (not your whole program!) of the *last built* (via `cargo build` or `cargo check`) RTIC application. The expanded code is not pretty printed by default, so you'll want to run `rustfmt` on it before you read it.

```
$ cargo build --example smallest --target thumbv7m-none-eabi
```
Shell

```
$ rustfmt target/rtic-expansion.rs
```
Shell

```
$ tail target/rtic-expansion.rs
```
Shell

```rust
#[doc = r" Implementation details"]
mod app {
    #[doc = r" Always include the device crate which contains the vector table"]
    use lm3s6965 as _;
    #[no_mangle]
    unsafe extern "C" fn main() -> ! {
        rtic::export::interrupt::disable();
        let mut core: rtic::export::Peripherals = core::mem::transmute(());
        core.SCB.scr.modify(|r| r | 1 << 1);
        rtic::export::interrupt::enable();
        loop {
            rtic::export::wfi()
        }
```
Rust

```
    }
}
```

3.10.5.2. Using `cargo-expand` tool

If not available, install:

```
$ cargo install cargo-expand
```

This sub-command will expand *all* the macros, including the `#[rtic::app]` attribute, and modules in your crate and print the output to the console.

```
# produces the same output as before
```
Shell

```
cargo expand --example smallest | tail
```
Shell

4. The magic behind Monotonics

Internally, all monotonics use a Timer Queue, which is a priority queue with entries describing the time at which their respective `Futures` should complete.

4.1. Implementing a `Monotonic` timer for scheduling

The `rtic-time` framework is flexible because it can use any timer which has compare-match and optionally supporting overflow interrupts for scheduling. The single requirement to make a timer usable with RTIC is implementing the `rtic-time::Monotonic` trait.

For RTIC 2.0, we assume that the user has a time library, e.g. `fugit`, as the basis for all time-based operations when implementing `Monotonic`. These libraries make it much easier to correctly implement the `Monotonic` trait, allowing the use of almost any timer in the system for scheduling.

The trait documents the requirements for each method. There are reference implementations available in `rtic-monotonics` that can be used for inspiration.

- `Systick based`, runs at a fixed interrupt (core tick) rate - simple, but comes with some overhead and can only be used while the core is running
- `RP2040 Timer`, a "proper" implementation with support for waiting for long periods without interrupts. Clearly demonstrates how to use the `TimerQueue` to handle scheduling.
- `nRF52 timers` implements monotonic & Timer Queue for the RTC and normal timers in nRF52's

Often, hardware counters only have a width of only 16 or 32 bit, meaning that they would overflow after some minutes to years, depending on the frequency of the counter peripheral. To overcome this issue, monotonic implementations for such counters must use an atomic overflow counter (e.g. `portable_atomic::AtomicU32`) to correctly calculate the current time instant. This leads to a race condition between the overflow counter and the time register, which isn't trivial to solve. Hence, a half period counter must be used. Please read and refer to `rtic_time::half_period_counter`, which provides helper methods and examples for implementing this logic.

4.2. Contributing

Contributing new implementations of `Monotonic` can be done in multiple ways:

- Implement the trait behind a feature flag in `rtic-monotonics`, and create a PR for them to be included in the main RTIC repository. This way, the implementations of are in-tree, RTIC can guarantee their correctness, and can update them in the case of a new release.
- Implement the changes in an external repository. Doing so will not have them included in `rtic-monotonics`, but may make it easier to do so in the future.

4.3. The timer queue

The timer queue is implemented as a list based priority queue, where list-nodes are statically allocated as part of the `Future` created when `await`-ing a Future created when waiting for the monotonic. Thus, the timer queue is infallible at run-time (its size and allocation are determined at compile time).

Similarly the channels implementation, the timer-queue implementation relies on a global *Critical Section* (CS) for race protection. For the examples a CS implementation is provided by a platform crate, for example `cortex-m/critical-section-single-core` or `esp32c3/critical-section`.

5. RTIC vs. the world

RTIC aims to provide the lowest level of abstraction needed for developing robust and reliable embedded software.

It provides a minimal set of required mechanisms for safe sharing of mutable resources among interrupts and asynchronously executing tasks. The scheduling primitives leverages on the underlying hardware for unparalleled performance and predictability, in effect RTIC provides in Rust terms a zero-cost abstraction to concurrent real-time programming.

5.1. Comparison regarding safety and security

Comparing RTIC to traditional a Real-Time Operating System (RTOS) is hard. Firstly, a traditional RTOS typically comes with no guarantees regarding system safety, even the most hardened kernels like the formally verified seL4 kernel. Their claims to integrity, confidentiality, and availability regards only the kernel itself (under additional assumptions its configuration and environment). They even state:

"An OS kernel, verified or not, does not automatically make a system secure. In fact, any system, no matter how secure, can be used in insecure ways." - seL4 FAQ

5.2. Security by design

In the world of information security we commonly find:

- confidentiality, protecting the information from being exposed to an unauthorized party,
- integrity, referring to accuracy and completeness of data, and
- availability, referring to data being accessible to authorized users.

Obviously, a traditional OS can guarantee neither confidentiality nor integrity, as both requires the security critical code to be trusted. Regarding availability, this typically boils down to the usage of system resources. Any OS that allows for dynamic allocation of resources, relies on that the application correctly handles allocations/de-allocations, and cases of allocation failures.

Thus their claim is correct, security is completely out of hands for the OS, the best we can hope for is that it does not add further vulnerabilities.

RTIC on the other hand holds your back. The declarative system wide model gives you a static set of tasks and resources, with precise control over what data is shared and between which parties. Moreover, Rust as a programming language comes with strong properties regarding integrity (compile time aliasing, mutability and lifetime guarantees, together with ensured data validity).

Using RTIC these properties propagate to the system wide model, without interference of other applications running. The RTIC kernel is internally infallible without any need of dynamically allocated data.

6. RTIC vs. Embassy

6.1. Differences

Embassy provides both Hardware Abstraction Layers, and an executor/runtime, while RTIC aims to only provide an execution framework. For example, embassy provides `embassy-stm32` (a HAL), and `embassy-executor` (an executor). On the other hand, RTIC provides the framework in the form of `rtic`, and the user is responsible for providing a PAC and HAL implementation (generally from the `stm32-rs` project).

Additionally, RTIC aims to provide exclusive access to resources at as low a level as possible, ideally guarded by some form of hardware protection. This allows for access to hardware without necessarily requiring locking mechanisms at the software level.

6.2. Mixing use of Embassy and RTIC

Since most Embassy and RTIC libraries are runtime agnostic, many details from one project can be used in the other. For example, using `rtic-monotonics` in an `embassy-executor` powered project works, and using `embassy-sync` (though `rtic-sync` is recommended) in an RTIC project works.

7. Awesome RTIC examples

See the `rtic-rs/rtic/examples` repository for complete examples.

Pull-requests are welcome!

8. Migrating from v1.0.x to v2.0.0

Migrating a project from RTIC `v1.0.x` to `v2.0.0` involves the following steps:

1. `v2.1.0` works on Rust Stable from 1.75 (**recommended**), while older versions require a `nightly` compiler via the use of `#![type_alias_impl_trait]`.
2. Migrating from the monotonics included in `v1.0.x` to `rtic-time` and `rtic-monotonics`, replacing `spawn_after`, `spawn_at`.
3. Software tasks are now required to be `async`, and using them correctly.
4. Understanding and using data types provided by `rtic-sync`.

For a detailed description of the changes, refer to the subchapters.

If you wish to see a code example of changes required, you can check out the full example migration page.

8.0.0.1. TL;DR (Too Long; Didn't Read)

1. Instead of `spawn_after` and `spawn_at`, you now use the `async` functions `delay`, `delay_until` (and related) with impls provided by `rtic-monotonics`.
2. Software tasks *must* be `async fn`s now. Not returning from a task is allowed so long as there is an `await` in the task. You can still `lock` shared resources.
3. Use `rtic_sync::arbiter::Arbiter` to `await` access to a shared resource, and `rtic_sync::channel::Channel` to communicate between tasks instead of `spawn`-ing new ones.

8.1. Migrating to `rtic-monotonics`

In previous versions of `rtic`, monotonics were an integral, tightly coupled part of the `#[rtic::app]`. In this new version, `rtic-monotonics` provides them in a more decoupled way.

The `#[monotonic]` attribute is no longer used. Instead, you use a `create_X_token` from `rtic-monotonics`. An invocation of this macro returns an interrupt registration token, which can be used to construct an instance of your desired monotonic.

`spawn_after` and `spawn_at` are no longer available. Instead, you use the async functions `delay` and `delay_until` provided by implementations of the `rtic_time::Monotonic` trait, available through `rtic-monotonics`.

Check out the code example for an overview of the required changes.

For more information on current monotonic implementations, see the `rtic-monotonics` documentation, and the examples.

8.2. Using async software tasks.

There have been a few changes to software tasks. They are outlined below.

8.2.0.1. Software tasks must now be async.

All software tasks are now required to be `async`.

8.2.0.1.1. Required changes.

All of the tasks in your project that do not bind to an interrupt must now be an `async fn`. For example:

```rust
#[task(
    local = [ some_resource ],
    shared = [ my_shared_resource ],
    priority = 2
)]
```

```
fn my_task(cx: my_task::Context) {
    cx.local.some_resource.do_trick();
    cx.shared.my_shared_resource.lock(|s| s.do_shared_thing());
}
```

becomes

```
#[task(                                                    Rust
    local = [ some_resource ],
    shared = [ my_shared_resource ],
    priority = 2
)]
async fn my_task(cx: my_task::Context) {
    cx.local.some_resource.do_trick();
    cx.shared.my_shared_resource.lock(|s| s.do_shared_thing());
}
```

8.2.1. Software tasks may now run forever

The new `async` software tasks are allowed to run forever, on one precondition: **there must be an `await` within the infinite loop of the task**. An example of such a task:

```
#[task(local = [ my_channel ] )]                           Rust
async fn my_task_that_runs_forever(cx: my_task_that_runs_forever::Context) {
    loop {
        let value = cx.local.my_channel.recv().await;
        do_something_with_value(value);
    }
}
```

8.2.2. `spawn_after` and `spawn_at` have been removed.

As discussed in the Migrating to `rtic-monotonics` chapter, `spawn_after` and `spawn_at` are no longer available.

8.3. Using `rtic-sync`

`rtic-sync` provides primitives that can be used for message passing and resource sharing in async context.

The important structs are: * The `Arbiter`, which allows you to await access to a shared resource in async contexts without using `lock`. * `Channel`, which allows you to communicate between tasks (both `async` and non-`async`).

For more information on these structs, see the `rtic-sync` docs

8.4. A complete example of migration

Below you can find the code for the implementation of the `stm32f3_blinky` example for v1.0.x and for v2.0.0. Further down, a diff is displayed.

8.4.1. v1.0.X

```
#![deny(unsafe_code)]                                      Rust
#![deny(warnings)]
```

```
#![no_main]
#![no_std]

use panic_rtt_target as _;
use rtic::app;
use rtt_target::{rprintln, rtt_init_print};
use stm32f3xx_hal::gpio::{Output, PushPull, PA5};
use stm32f3xx_hal::prelude::*;
use systick_monotonic::{fugit::Duration, Systick};

#[app(device = stm32f3xx_hal::pac, peripherals = true, dispatchers = [SPI1])]
mod app {
    use super::*;

    #[shared]
    struct Shared {}

    #[local]
    struct Local {
        led: PA5<Output<PushPull>>,
        state: bool,
    }

    #[monotonic(binds = SysTick, default = true)]
    type MonoTimer = Systick<1000>;

    #[init]
    fn init(cx: init::Context) -> (Shared, Local, init::Monotonics) {
        // Setup clocks
        let mut flash = cx.device.FLASH.constrain();
        let mut rcc = cx.device.RCC.constrain();

        let mono = Systick::new(cx.core.SYST, 36_000_000);

        rtt_init_print!();
        rprintln!("init");

        let _clocks = rcc
            .cfgr
            .use_hse(8.MHz())
            .sysclk(36.MHz())
            .pclk1(36.MHz())
            .freeze(&mut flash.acr);

        // Setup LED
        let mut gpioa = cx.device.GPIOA.split(&mut rcc.ahb);
```

```
        let mut led = gpioa
            .pa5
            .into_push_pull_output(&mut gpioa.moder, &mut gpioa.otyper);
        led.set_high().unwrap();

        // Schedule the blinking task
        blink::spawn_after(Duration::<u64, 1, 1000>::from_ticks(1000)).unwrap();

        (
            Shared {},
            Local { led, state: false },
            init::Monotonics(mono),
        )
    }

    #[task(local = [led, state])]
    fn blink(cx: blink::Context) {
        rprintln!("blink");
        if *cx.local.state {
            cx.local.led.set_high().unwrap();
            *cx.local.state = false;
        } else {
            cx.local.led.set_low().unwrap();
            *cx.local.state = true;
        }
        blink::spawn_after(Duration::<u64, 1, 1000>::from_ticks(1000)).unwrap();
    }
}
```

8.4.2. V2.0.0

Rust

```
#![deny(unsafe_code)]
#![deny(warnings)]
#![no_main]
#![no_std]

use panic_rtt_target as _;
use rtic::app;
use rtic_monotonics::systick::prelude::*;
use rtt_target::{rprintln, rtt_init_print};
use stm32f3xx_hal::gpio::{Output, PushPull, PA5};
use stm32f3xx_hal::prelude::*;

systick_monotonic!(Mono, 1000);

#[app(device = stm32f3xx_hal::pac, peripherals = true, dispatchers = [SPI1])]
mod app {
```

```
17      use super::*;
18
19      #[shared]
20      struct Shared {}
21
22      #[local]
23      struct Local {
24          led: PA5<Output<PushPull>>,
25          state: bool,
26      }
27
28      #[init]
29      fn init(cx: init::Context) -> (Shared, Local) {
30          // Setup clocks
31          let mut flash = cx.device.FLASH.constrain();
32          let mut rcc = cx.device.RCC.constrain();
33
34          // Initialize the systick interrupt & obtain the token to prove that we
            did
35          Mono::start(cx.core.SYST, 36_000_000); // default STM32F303 clock-rate
            is 36MHz
36
37          rtt_init_print!();
38          rprintln!("init");
39
40          let _clocks = rcc
41              .cfgr
42              .use_hse(8.MHz())
43              .sysclk(36.MHz())
44              .pclk1(36.MHz())
45              .freeze(&mut flash.acr);
46
47          // Setup LED
48          let mut gpioa = cx.device.GPIOA.split(&mut rcc.ahb);
49          let mut led = gpioa
50              .pa5
51              .into_push_pull_output(&mut gpioa.moder, &mut gpioa.otyper);
52          led.set_high().unwrap();
53
54          // Schedule the blinking task
55          blink::spawn().ok();
56
57          (Shared {}, Local { led, state: false })
58      }
59
60      #[task(local = [led, state])]
```

```
61      async fn blink(cx: blink::Context) {
62          loop {
63              rprintln!("blink");
64              if *cx.local.state {
65                  cx.local.led.set_high().unwrap();
66                  *cx.local.state = false;
67              } else {
68                  cx.local.led.set_low().unwrap();
69                  *cx.local.state = true;
70              }
71              Mono::delay(1000.millis()).await;
72          }
73      }
74  }
```

8.4.3. A diff between the two projects

Note: This diff may not be 100% accurate, but it displays the important changes.

Diff

```
 #![no_main]
 #![no_std]

 use panic_rtt_target as _;
 use rtic::app;
 use stm32f3xx_hal::gpio::{Output, PushPull, PA5};
 use stm32f3xx_hal::prelude::*;
-use systick_monotonic::{fugit::Duration, Systick};
+use rtic_monotonics::Systick;

 #[app(device = stm32f3xx_hal::pac, peripherals = true, dispatchers = [SPI1])]
 mod app {
@@ -20,16 +21,14 @@ mod app {
         state: bool,
     }

-    #[monotonic(binds = SysTick, default = true)]
-    type MonoTimer = Systick<1000>;
-
     #[init]
     fn init(cx: init::Context) -> (Shared, Local, init::Monotonics) {
         // Setup clocks
         let mut flash = cx.device.FLASH.constrain();
         let mut rcc = cx.device.RCC.constrain();

-        let mono = Systick::new(cx.core.SYST, 36_000_000);
+        let mono_token = rtic_monotonics::create_systick_token!();
+        let mono = Systick::start(cx.core.SYST, 36_000_000, mono_token);
```

```
         let _clocks = rcc
             .cfgr
@@ -46,7 +45,7 @@ mod app {
         led.set_high().unwrap();

         // Schedule the blinking task
-        blink::spawn_after(Duration::<u64, 1, 1000>::from_ticks(1000)).unwrap();
+        blink::spawn().unwrap();

         (
             Shared {},
@@ -56,14 +55,18 @@ mod app {
     }

     #[task(local = [led, state])]
-    fn blink(cx: blink::Context) {
-        rprintln!("blink");
-        if *cx.local.state {
-            cx.local.led.set_high().unwrap();
-            *cx.local.state = false;
-        } else {
-            cx.local.led.set_low().unwrap();
-            *cx.local.state = true;
-        blink::spawn_after(Duration::<u64, 1, 1000>::from_ticks(1000)).unwrap();
-    }
+    async fn blink(cx: blink::Context) {
+        loop {
+            // A task is now allowed to run forever, provided that
+            // there is an `await` somewhere in the loop.
+            SysTick::delay(1000.millis()).await;
+            rprintln!("blink");
+            if *cx.local.state {
+                cx.local.led.set_high().unwrap();
+                *cx.local.state = false;
+            } else {
+                cx.local.led.set_low().unwrap();
+                *cx.local.state = true;
+            }
+        }
+    }
 }
```

9. Under the hood

This is chapter is currently work in progress, it will re-appear once it is more complete

This section describes the internals of the RTIC framework at a *high level*. Low level details like the parsing and code generation done by the procedural macro (`#[app]`) will not be explained here. The focus will be the analysis of the user specification and the data structures used by the runtime.

We highly suggest that you read the embedonomicon section on concurrency before you dive into this material.

9.1. Target Architecture

9.1.1. Cortex-M Devices

While RTIC can currently target all Cortex-m devices there are some key architecture differences that users should be aware of. Namely, the absence of Base Priority Mask Register (`BASEPRI`) which lends itself exceptionally well to the hardware priority ceiling support used in RTIC, in the ARMv6-M and ARMv8-M-base architectures, which forces RTIC to use source masking instead. For each implementation of lock and a detailed commentary of pros and cons, see the implementation of lock in src/export.rs.

These differences influence how critical sections are realized, but functionality should be the same except that ARMv6-M/ARMv8-M-base cannot have tasks with shared resources bound to exception handlers, as these cannot be masked in hardware.

Table 1 below shows a list of Cortex-m processors and which type of critical section they employ.

Table 1: Critical Section Implementation by Processor Architecture

Processor	Architecture	Priority Ceiling	Source Masking
Cortex-M0	ARMv6-M		✓
Cortex-M0+	ARMv6-M		✓
Cortex-M3	ARMv7-M	✓	
Cortex-M4	ARMv7-M	✓	
Cortex-M7	ARMv7-M	✓	
Cortex-M23	ARMv8-M-base		✓
Cortex-M33	ARMv8-M-main	✓	

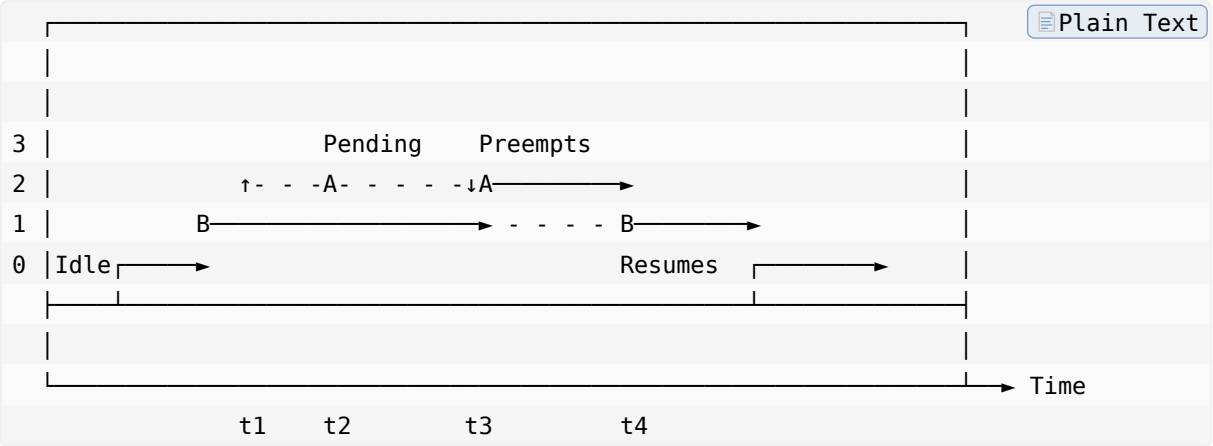
9.1.1.1. Priority Ceiling

This is covered by the Resources page of this book.

9.1.1.2. Source Masking

Without a `BASEPRI` register which allows for directly setting a priority ceiling in the Nested Vectored Interrupt Controller (NVIC), RTIC must instead rely on disabling (masking) interrupts. Consider Figure 1 below, showing two tasks A and B where A has higher priority but shares a resource with B.

Figure 1: Shared Resources and Source Masking



At time *t1*, task B locks the shared resource by selectively disabling (using the NVIC) all other tasks which have a priority equal to or less than any task which shares resources with B. In effect this creates a virtual priority ceiling, mirroring the `BASEPRI` approach. Task A is one such task that shares resources with task B. At time *t2*, task A is either spawned by task B or becomes pending through an interrupt condition, but does not yet preempt task B even though its priority is greater. This is because the NVIC is preventing it from starting due to task A being disabled. At time *t3*, task B releases the lock by re-enabling the tasks in the NVIC. Because task A was pending and has a higher priority than task B, it immediately preempts task B and is free to use the shared resource without risk of data race conditions. At time *t4*, task A completes and returns the execution context to B.

Since source masking relies on use of the NVIC, core exception sources such as HardFault, SVCall, PendSV, and SysTick cannot share data with other tasks.

9.1.2. RISC-V Devices

All the current RISC-V backends work in a similar way as Cortex-M devices with priority ceiling. Therefore, the Resources page of this book is a good reference. However, some of these backends are not full hardware implementations, but use software to emulate a physical interrupt controller. Therefore, these backends do not implement hardware tasks, and only software tasks are needed. Furthermore, the number of software tasks for these targets is not bounded by the number of available physical interrupt sources.

Table 2 below compares the available RISC-V backends.

Table 2: Critical Section Implementation by Processor Architecture

Backend	Compatible targets	Backend-specific configuration	Hardware Tasks	Software Tasks	Number of tasks bounded by HW
`riscv-esp32c3-backend`	ESP32-C3 only		✓	✓	✓
`riscv-mecall-backend`	Any RISC-V device			✓	
`riscv-clint-backend`	Devices with CLINT peripheral	✓		✓	

9.1.2.1. `riscv-mecall-backend`

It is not necessary to provide a list of dispatchers in the `#[app]` attribute, as RTIC will generate them at compile time. Priority levels can go from 0 (for the `idle` task) to 255.

9.1.2.2. `riscv-clint-backend`

It is not necessary to provide a list of `dispatchers` in the `#[app]` attribute, as RTIC will generate them at compile time. Priority levels can go from 0 (for the `idle` task) to 255.

You **must** include a `backend`-specific configuration in the `#[app]` attribute so RTIC knows the ID number used to identify the HART running your application. For example, for `e310x` chips, you would configure a minimal application as follows:

```rust
#[rtic::app(device = e310x, backend = H0)]
mod app {
  // your application here
}
```

In this way, RTIC will always refer to HART `H0`.